

Final Year Design Project

College Management System
for Govt. Graduate College, Mandi Bahauddin



By

Hadia
085668

Syeda Laraib Qamar Kazmi
085713

Sharfa Kiran
085646

Under the supervision of
Prof. Ubaid Ullah

Bachelor of Science in Information Technology (2022-2026)

DEPARTMENT OF INFORMATION TECHNOLOGY
GOVT. GRADUATE COLLEGE, MANDI BAHAUDDIN

College Management System for Govt. Graduate College, Mandi Bahauddin

A project presented to
Govt. Graduate College, Mandi Bahauddin

In partial fulfilment
of the requirement for the degree of

Bachelor of Science in Information Technology (2022-2026)

By

Hadia 085668

Syeda Laraib Qamar Kazmi 085713

Sharfa Kiran 085646

**DEPARTMENT OF INFORMATION TECHNOLOGY
GOVT. GRADUATE COLLEGE, MANDI BAHAUDDIN**

Declaration

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted as part of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: _____

Hadia [085668]

Signature: _____

Sharfa Kiran [085646]

Signature: _____

Syeda Laraib Qamar Kazmi [085713]

Certificate of Approval

It is to certify that the final year project (FYP) of BSIT, session (2022-2026), titled "College Management System for Govt. Graduate College, Mandi Bahauddin" was developed by Hadia (2022-mcm-359), Syeda Laraib Qamar Kazmi (2022-mcm-37), and Sharfa Kiran (2022-mcm-84) under the supervision of Prof. Ubaid Ullah; in my opinion, it is fully adequate, in scope and quality, for the degree of Bachelor of Science in Information Technology.

Signature: _____

FYDP Advisor: Prof. Ubaid Ullah

Signature: _____

Head of FYDP Coordination Office: Prof. Muhammad Faiyaz

Signature: _____

Dated: _____

Head of Department, Information Technology: Prof. Muhammad Faiyaz

Executive Summary

Govt. Graduate College, Mandi Bahauddin runs its academic life on paper. Roughly 3,000 students across Intermediate and BS programmes, and about 50 faculty members, still queue at the accounts window for a fee challan, wait for a hand-written attendance register to be tallied, and read their semester result off a notice board instead of a phone. Every one of those steps is somewhere a form can go missing, a number can be mistyped, or a student simply doesn't find out in time, a wrong CGPA on a transcript, a challan that doesn't match what the accounts office has on file, an exam schedule that reaches half the class.

The system we built is six applications, not one. There is an admin app, a teacher app, and a student app, and each ships both as an Android phone app and as a Compose-for-Desktop app. All six sit on a single shared Supabase database and draw from one shared component library, the "Modernist" red/ink design system, so a student's attendance, a session's fee structure, or a published datesheet is the same record no matter which app opens it, on either platform. The admin side holds the college's structure: departments, the academic sessions inside them, and each session's curriculum, timetable, and fee heads. Teachers work against that same structure, marking attendance, entering midterm and sessional marks, submitting exam papers, and recording semester GPA/CGPA. Students see only their own slice of it, attendance, marks, fee challan, datesheet, and calendar notices, without walking into an office.

The reason so much of this works cleanly is that the access rules live in the database, not in the apps. Postgres on Supabase stores everything; Row-Level Security decides who can read and write each row; GoTrue handles sign-in; and Storage holds uploaded files such as exam papers. That was a deliberate switch away from an earlier Firebase/Firestore prototype, and the payoff is the guarantees a relational schema gives that a document store could not, a mark cannot be attributed to a student who was never enrolled in that session, and a fee record cannot outlive the session it belongs to, because foreign keys and row-level policies refuse to let it. The apps are Jetpack Compose on mobile and Compose for Desktop on the desktop, both built on the same MVVM/Repository split and wired with Hilt, which is what lets all six share one component library rather than maintaining six separate sets of screens.

What this report describes is the system as it stands in the codebase. Its spine is a department-centric academic structure with per-session fees and a lock-then-approve workflow for marks: a teacher enters a score once, and changing it afterward requires an admin's sign-off through a mark-correction request. Around that sit semester results with GPA/CGPA and supply-subject tracking, fines, a calendar, datesheets, and a tiered Insights view built on Postgres views, an admin sees the college-wide figures while a teacher sees only their own classes, and that scoping is the database's own row-level rules doing the work, not extra logic in the app. All six applications build and run against the live Supabase project.

Keywords: College Management System, Row-Level Security, Jetpack Compose, Supabase, Android

Acknowledgement

We are grateful to our supervisor, Prof. Ubaid Ullah, for his guidance throughout this project, and to the Department of Information Technology at Govt. Graduate College, Mandi Bahauddin, for its support. We also thank the college staff who explained the day-to-day academic and office processes the system was built around, and our families for their encouragement.

Abbreviations

Abbreviation	Description
AGP	Android Gradle Plugin
API	Application Programming Interface
APK	Android Package (installable app)
CGPA	Cumulative Grade Point Average
CMS	College Management System
CSV	Comma-Separated Values
DI	Dependency Injection
DTO	Data Transfer Object
FR	Functional Requirement
FT	Functional Test
GPA	Grade Point Average
IT	Integration Test
JDK	Java Development Kit
MSI	Microsoft Installer package
MVVM	Model-View-ViewModel
PDF	Portable Document Format
PT	Performance Test
RLS	Row-Level Security
SDK	Software Development Kit
UC	Use Case
UI	User Interface
UT	Unit Test
XLSX	Excel Spreadsheet format

Table of Contents

Declaration	3
Certificate of Approval.....	4
Executive Summary	5
Acknowledgement.....	6
Abbreviations	7
Table of Contents	8
1. Introduction	10
1.1 Problem Statement.....	10
1.2 Problem Solution	10
1.3 Objectives of the Proposed System.....	11
1.4 Scope.....	12
1.5 System Components.....	13
1.6 Related System Analysis / Literature Review	15
1.7 Vision Statement.....	16
1.8 System Limitations and Constraints	16
1.9 Tools and Technologies	17
1.10 Project Deliverables	18
1.11 Project Planning	19
1.12 Summary	19
2. Analysis	19
2.1 User Classes and Characteristics	20
2.2 Requirement Identifying Technique	20
2.2.3 Use Case Descriptions	21
2.3 Functional Requirements	40
2.4 Non-Functional Requirements	64
2.5 External Interface Requirements.....	70
2.6 Summary	74
3. System Design	74
3.1 Design Considerations	74
3.2 Design Models	76

3.3 Architectural Design	78
3.4 Data Design.....	80
3.5 User Interface Design	83
3.6 Behavioural Model.....	87
3.7 Design Decisions	89
3.8 Summary	90
4. Implementation.....	91
4.1 Algorithm.....	91
4.2 External APIs/SDKs	98
4.3 Code Repository.....	100
4.4 Summary	103
5. Testing and Evaluation	103
5. Introduction.....	103
5.1 Unit Testing (UT)	103
5.2 Functional Testing (FT)	105
5.3 Integration Testing (IT).....	107
5.4 Performance Testing (PT).....	109
5.5 Summary	110
6. System Conversion.....	111
6. Introduction.....	111
6.1 Conversion Method.....	111
6.2 Deployment.....	112
6.3 Post Deployment Testing.....	116
6.4 Challenges.....	117
6.5 Summary	118
7. Conclusion	119
7. Introduction.....	119
7.1 Evaluation	119
7.2 Traceability Matrix	121
7.3 Conclusion	123
7.4 Future Work	124
Appendix A: Fully Dressed Use Case Descriptions.....	126

Appendix B: General Coding Standards and Guidelines	145
Appendix C: Application Prototype	147

1. Introduction

Before the later chapters get into requirements, design, implementation, testing, and deployment, this chapter sets the ground the rest of the report stands on. It begins with the problem at Govt. Graduate College, Mandi Bahauddin, an academic routine still run largely on paper, and the solution that answers it, then fixes the objectives and scope that everything afterward is measured against. From there it introduces the six applications that make up the system: an admin, a teacher, and a student app, each shipped on both Android and desktop. It places those against the related systems already in use, and closes with the project's vision, the limitations and constraints it works within, and the tools, deliverables, and plan behind the build.

1.1 Problem Statement

Govt. Graduate College, Mandi Bahauddin runs on paper. Around 3,000 students across its Intermediate and BS programmes, and roughly 50 faculty members, depend on class registers, printed notices, and trips to administrative offices for work that repeats every term. Attendance sits in a register until someone tallies it by hand, so a student's real standing is not known until the counting is done. Marks and semester results move through several paper steps, which makes them slow to check, slow to correct, and slow to share, and easy to get wrong, whether that surfaces as a mistyped CGPA on a transcript or a fee challan that does not match what the accounts office has on file. Timetables, examination dates, and notices live on a noticeboard, so a student who is absent or simply does not walk past it can miss them; an exam schedule that reaches only half the class is a real outcome here, not a hypothetical. For anything about fees, fines, results, or schedules, students often have to come to campus and stand at a window to ask. Teachers carry their own version of this, copying the same figures once into the class record and again into an administrative report. The administration's difficulty is that the picture is scattered: departments, academic sessions, teachers, and students each sit in separate records, so assembling a complete report means comparing registers by hand, and a correction made in one place rarely reaches the others. Paper is also hard to search and no use at all to anyone away from campus. None of this is unusual for a college of this size, it just grows heavier as the number of students, classes, and activities increases. What is missing is a single coordinated system that administrators, teachers, and students can each work in, and see only their own part of, from either a phone or a desktop.

1.2 Problem Solution

The answer is one College Management System split across six applications: an admin, a teacher, and a student app, each built for both a phone and a desktop. They are less six separate programs than six views of the same college records. Everything sits in a single shared database, so a figure entered once by whoever is responsible for it becomes the record everyone else works from, instead of being written down a second time in another register. That alone removes most

of what makes §1.1 painful, the manual re-tallying, the correction that never reaches the other book, the report assembled by comparing registers side by side. Sign-in establishes who you are and opens only your part of the system, and that boundary holds all the way down to the data: each role can read and change exactly what it is allowed to and nothing more, because the database itself enforces the rule rather than trusting each app to behave.

The admin apps hold the college's structure and oversight, departments, academic sessions, students, teachers, other administrators, timetables, fees, attendance reports, examinations, and notices. The teacher apps cover the daily academic work: viewing assigned classes, marking attendance, entering marks, submitting examination papers, checking schedules, and requesting a correction when a recorded mark has to change. The student apps give each student a clear read on their own attendance, timetable, marks and results, fee challan, datesheets, and notices, and let a newly registered student ask for their account to be linked to the correct college record.

The two formats are meant to complement each other, not fork into separate procedures over separate data. A phone is what a teacher has in hand during class and what a student has when away from campus; the desktop gives the same role-based work a larger screen for office data entry and printing, such as generating a fee challan. Where an action carries weight, changing a mark that was already recorded, or approving a student's link request, it runs through a visible approval step rather than a silent edit. That is what keeps the records trustworthy, not just quicker to produce. Taken together, the aim is straightforward: college information that is easier to keep correct, faster to reach, and dependable for administrators, teachers, and students alike.

1.3 Objectives of the Proposed System

Each objective below is concrete enough to check against the finished system:

- Ship six working applications, a mobile and a desktop app each for administrators, teachers, and students, all running against the same college database.
- Hold a single authoritative record for every department, academic session, subject, teacher, student, timetable, attendance entry, mark, semester result, and fee structure, so the same value is never entered twice.
- Let an authorised teacher record a full class attendance register on screen instead of on a paper register, and make that register visible to the administration and to the affected students once it syncs.
- Route every change to an already-recorded mark through an administrator's approval, so a submitted score cannot be altered silently.
- Record semester results, including GPA, CGPA, and supply (repeat) subjects, against the enrolled student rather than on a separate sheet.
- Give each student their own attendance, timetable, marks, results, fee challan, fines, datesheets, and notices on their own device, so routine questions no longer require an office visit.
- Publish notices, calendar events, and examination datesheets to the roles they are meant for, on both mobile and desktop.

- Show administrators live summaries of departments, sessions, teachers, students, attendance, and academic performance drawn straight from the records rather than compiled by hand, with a teacher's view scoped to their own classes by the same rules.
- Enforce role-based access throughout, so each user can see and do only what their role allows.
- Keep the mobile and desktop version of each role consistent in content, navigation, and the main workflows.
- Verify all six applications through functional and user-interface testing before final submission.

1.4 Scope

The system covers the everyday academic and administrative work of Govt. Graduate College, Mandi Bahauddin for three groups of people: administrators, teaching staff, and enrolled students. Each group has both a mobile and a desktop application, and both work on the same college records, so what a person can see and do depends on their role, not on the device in their hand.

In scope:

- Setting up and maintaining the college's structure, departments, academic sessions, subjects, classes and offerings, teachers, administrators, students, and the requests students raise to link an account to their own record.
- Class timetables and teacher schedules, examination datesheets, and college calendar events.
- Attendance: teachers record a class register, students see their own attendance, and administrators read it back as reports.
- Marks entry, administrator-approved mark corrections, semester results with GPA/CGPA and supply subjects, examination-paper submission, and the academic summaries built from all of this.
- Session fee structures, per-student fee challans, fines, notices, and notifications.
- Mobile and desktop access for all three roles, with the same information and role-appropriate actions on each.

Out of scope:

- Online fee payment or any direct link to a bank. The system shows the fee structure and produces the challan; the student still pays at the accounts office.
- Automatic grading of examination papers, or any software standing in for a teacher's academic judgement.
- Public website features, online admissions, hostel management, payroll, and library circulation.
- iPhone apps, browser-based access, and running more than one college from a single installation, the build targets Android phones and the desktop, for this one college.

1.5 System Components

The system presents six user-facing applications: an admin, a teacher, and a student app, each in a mobile (Android) and a desktop version. The two versions for a given role carry the same information and follow the same college procedures, only the layout changes to suit the screen, so the phone leans on compact cards and lists while the desktop spreads the same records into wider tables.

That consistency is not kept up by hand across six separate codebases. The apps stand on a shared foundation: a portable core module holds the domain model, the data-transfer objects, the repositories, and the authentication that every app relies on, and two platform kits build on top of it, mobile-shared (the Android UI kit, together with the local cache and dependency wiring the three phone apps use) and desktop-shared (the Compose-for-Desktop UI kit the three desktop apps use). Because the shared logic lives in one place, a rule written once holds everywhere it is applied, and each of the six apps is left to assemble its own screens from common parts.

The applications and their individual features are described in the subsections that follow.

1.5.1 Admin Mobile Application

The college's control panel in a pocket, the full academic structure plus the approvals and reports an administrator needs while away from the office desk:

- **Dashboard** with the headline totals (students, teachers, departments), pending requests, recent notifications, and shortcuts into the common tasks.
- **Academic structure:** departments and their detail, academic sessions, per-semester subjects (the curriculum), class rosters, per-session and master timetables, buildings and rooms, and each session's fee structure.
- **People:** teachers and their individual permissions, other administrators, and student profiles, including issuing and tracking a student's fines.
- **Approvals:** reviewing student account-link requests and teachers' mark-correction requests before either takes effect.
- **Records and reporting:** attendance reports, an Insights view of academic summaries, submitted examination papers, and examination datesheets.
- **Communication:** calendar events and notifications, both viewing them and sending them.

1.5.2 Admin Desktop Application

The same administrator, at a desk instead of on a phone, identical data and rules, laid out for a larger screen and the office work that goes with it:

- Covers the same administrative areas and permissions as the Admin Mobile app, academic structure, people, approvals, attendance and records, the Insights summaries, calendar, notifications, and datesheets, all working on the same college data.

- Lays those records out for a large screen: wide, multi-column data tables built for working through long lists of departments, sessions, teachers, students, and records without the constant scrolling a phone forces.
- Adds the desk-bound work a phone handles poorly, exporting attendance and academic reports to CSV or PDF, generating and printing fee challans, and opening and saving files through the desktop's own file dialogs.
- Keeps the approval steps, status labels, navigation order, and data identical to the mobile app, so moving between the two never changes the rules or the numbers.

1.5.3 Teacher Mobile Application

Built for a teacher's day, mostly attendance and marks, with the surrounding academic work kept close at hand:

- **Home** with the next class, upcoming deadlines, recent notices, and shortcuts into the common tasks.
- **Attendance:** mark a full class register for the day, and open the history to check or review earlier registers.
- **Marks:** enter midterm and sessional scores; once a score is locked, request a correction that an administrator must approve rather than editing it directly.
- **Semester results:** record a student's GPA, CGPA, and supply subjects for the term.
- **Examination papers:** submit exam papers through the Exams hub.
- **Schedule and classes:** the teacher's own timetable, the students assigned to them, the published examination datesheet, and college calendar events.
- **Insights**, scoped to the teacher's own classes, the same kind of summaries the admin sees college-wide, limited to what this teacher is responsible for.
- **Notifications**, plus reviewing student link requests and sending notifications where the administrator has granted those permissions.
- **Profile** and account.

1.5.4 Teacher Desktop Application

The same teaching work as the mobile app, on a desk-sized screen for the parts that involve long lists and files:

- Covers the same teaching responsibilities and access limits as the Teacher Mobile app, attendance and history, marks entry with locked-mark correction requests, semester results, exam-paper submission, schedule, assigned students, datesheets, calendar, the class-scoped Insights, and notifications, all on the same college data.
- Uses the extra screen space for the list-heavy work: class rosters, attendance registers, marks-entry grids, schedules, and result review sit in wide tables rather than scrolling cards.

- Handles the desk-bound file work, choosing and opening files, such as the PDF for an exam-paper submission, and printing where it is useful.
- Keeps the teacher's classes, records, notices, and approval status identical to the mobile version, so nothing shifts when moving between the two.

1.5.5 Student Mobile Application

A student's own record in their pocket, reachable without an office or noticeboard visit:

- **Register and link:** a student registers, then requests that the account be linked to their college record; the rest of the app stays gated until an administrator approves that link.
- **Home** with the day's academic information, the next class, and recent notices.
- **Academics:** personal attendance summary, timetable, marks, semester results (GPA/CGPA), the published datesheet, and the college calendar.
- **Fees:** the fee challan, which the student can open and download as a PDF, with any fines shown on their profile.
- **Notifications and profile,** the profile also carrying enrolment history.

1.5.6 Student Desktop Application

The student's own information on a full screen, with the printing the college still asks for on paper:

- Shows the same personal academic and fee information as the Student Mobile app, attendance, timetable, marks, semester results, datesheet, calendar, fee challan, and notifications, with fines on the profile, behind the same account-linking gate.
- Uses the larger display for easier reading of timetables, attendance history, results, and datesheets.
- Handles the desk-bound file work: downloading and printing the fee challan, and saving or printing the student's own records where useful.
- Keeps account and link status, notifications, and displayed information identical to the mobile version.

1.6 Related System Analysis / Literature Review

Putting academic information online is not new, learning platforms and college portals already do it, and their value is clear. What most of them do not do is connect all three sides of a college at once. A given tool is usually built for classroom teaching or for office administration, for one device type or one role, rather than giving administrators, teachers, and students matching mobile and desktop apps over the same records. Table 1-1 sets the closest comparisons against what this system does differently.

Table 1-1: Related System Analysis with Proposed Project Solution

Related system	Observed weakness	How this system responds
----------------	-------------------	--------------------------

Related system	Observed weakness	How this system responds
Google Classroom	Handles course communication and learning material well, but is not built to run a college's departments, academic sessions, fees, attendance reporting, or administrative approvals.	Keeps that academic and administrative record-keeping at its centre, departments, sessions, rosters, fees, attendance reports, and a lock-then-approve workflow for mark changes, while still carrying notices, datesheets, and a calendar through to each role.
General college-management portals	Tend to focus on office use or browser access, and often lack matching role-based applications for teachers and students across both mobile and desktop.	Ships six coordinated applications, an admin, a teacher, and a student app, each on Android and desktop, so every role gets a fitting experience on the device in front of them, all reading the same data.
Paper registers and noticeboards	Need physical access, repeated copying, manual totalling, and separate re-checking whenever something changes.	Records each figure once, makes an approved change visible to the affected users, and keeps a clear, named responsibility for the actions that matter.

1.7 Vision Statement

For the administrators, teachers, and students of Govt. Graduate College, Mandi Bahauddin, the College Management System replaces separate paper records, office visits, and noticeboard checks with one coordinated set of six role-based applications, an admin, a teacher, and a student app, each on both a phone and a desktop. It holds the college's administration, the daily work of teaching, and each student's own academic, fee, examination, and notice information on the same shared records, so a figure entered once is the figure everyone entitled to see it works from. Where the present process makes people chase information across registers and windows, or offers only one device or one slice of college life, this system sets out to give all three groups timely, dependable access to the same approved records through whichever device suits the moment.

1.8 System Limitations and Constraints

Limitations, factors outside the team's control that bound what the system can do:

- The apps need a network connection for anything live. Each app keeps a local cache and still shows its last-synced data when offline, but receiving the newest records and submitting new or changed ones both require connectivity.
- Hosting and uptime depend on Supabase, the external service that stores and exchanges the college's data; an outage there is beyond the team's control.
- The Android apps require Android 7.0 (API 24) or later, and the desktop apps need a Java-capable computer; older phones are not supported.
- A report is only as accurate as what authorised users enter, the system cannot correct information it was never given.

Constraints, choices the team made deliberately:

- The first release targets a single college and follows the academic practices of Govt. Graduate College, Mandi Bahauddin.
- The apps are built for Android phones and a Windows desktop (on the JVM); iPhone and browser versions were left out of scope.
- Fees are informational only. Students can see the fee structure and challan, but payment stays with the college's existing process, no payment gateway is integrated.
- Examination papers are submitted and managed in the system, but grading stays with the teacher; there is no automatic marking.
- The scope and depth of testing were bounded by the academic schedule, the team's size, and the devices and sample data available during development.

1.9 Tools and Technologies

All six applications are written in one language on one build, which is what lets them share their logic and their look rather than being six separate projects. The stack is Kotlin throughout, with Jetpack Compose for the Android apps and Compose Multiplatform for the desktop apps, Hilt for dependency wiring, Room for the on-device cache, and a Supabase (PostgreSQL) backend reached through the Supabase Kotlin SDK. Table 1-2 lists the principal tools, their versions, and why each is used.

Table 1-2: Tools and Technologies for the Proposed Project

Tool / Technology	Version	Rationale
Kotlin	2.1.0	The single language for all six apps and the shared modules, so common logic is written once.
Jetpack Compose (BOM)	2024.12.01	Declarative UI toolkit for the three Android applications.
Compose Multiplatform	1.7.3	Builds the three desktop UIs with the same Compose approach, reusing the shared

Tool / Technology	Version	Rationale
		component kit.
Hilt	2.52	Dependency injection that wires repositories and view models across the apps.
Room	2.6.1	On-device database that caches records, so each app still shows data offline and syncs when back online.
Supabase Kotlin SDK	3.1.4	Client for the backend, sign-in (auth/GoTrue), record access (Postgrest), and file storage.
PostgreSQL	Supabase-managed	Relational store for every college record; Row-Level Security enforces who may read and write each row.
Ktor	3.1.1	HTTP engine the Supabase client runs on.
Android SDK	compile/target 36, min 24	Builds and runs the mobile apps; supports Android 7.0 (API 24) and later.
JDK	17	Build and runtime toolchain for the desktop apps and the shared code.
Gradle (with AGP 8.13.2)	8.13	Repeatable multi-module build and packaging for all six applications.
Git & GitHub	N/A	Version control and team collaboration.

1.10 Project Deliverables

The project hands over:

- This report and its appendices, the fully dressed use cases (Appendix A), the coding standards the project follows (Appendix B), and the application prototype (Appendix C), along with the planning, requirements, and design material and the test records behind them.
- Six installable applications: the Admin, Teacher, and Student apps as Android packages, and the same three as Compose-for-Desktop builds.

- The configured Supabase project that backs them, the database schema, the Row-Level Security policies, and the file-storage buckets.
- The complete source for all six apps and the shared modules they are built from (core, mobile-shared, and desktop-shared).

1.11 Project Planning

The work is planned in phases, each one a milestone that has to hold up before the next can build on it. The phases are defined by what depends on what, not by the calendar:

- **Foundation**, requirements and design, the shared data model and sign-in, and the academic structure (departments, sessions, subjects, and rosters) that every later feature attaches to.
- **Core academic operations**, attendance, timetables, marks entry with the lock-then-approve correction path, and semester results. These rest on the foundation, since a mark or an attendance row has to point at a real enrolled student.
- **Surrounding services**, fees and challans, fines, notices and notifications, the calendar, datesheets, exam-paper submission, and the Insights summaries drawn from the academic records.
- **Desktop parity**, the three desktop apps, reusing the shared logic to match the mobile apps feature for feature rather than re-implementing anything.
- **Verification and handover**, functional, integration, and performance testing across all six apps, college-data setup, and the report with its appendices.

Testing, data setup, and documentation are not held back for a final phase; they run against each milestone as it lands, so a phase counts as done only once its features are verified. Figure 1-1 lays out these phases and their milestones.

1.12 Summary

Govt. Graduate College still runs its records by hand, and closing that gap is what this project is for: one coordinated system of six applications, an admin, a teacher, and a student app, each on mobile and desktop, over a single shared set of records. This chapter set out that problem and solution, the objectives and scope the system is measured against, and its components, limitations, tools, and plan of work. Set against the related systems, the real contribution is not any one feature but the joining-up itself, routine college work connected across three roles and two device types, on the same approved data. These foundations carry into the requirements in Chapter 2 and give the yardstick the finished system is later judged by.

2. Analysis

Where the last chapter set out what the system is for, this one pins down what it must actually do and the conditions it has to hold to. It begins with the people who use the six applications, the three roles across mobile and desktop, and how their needs become use cases and, from those, concrete requirements. The functional requirements spell out the services each role gets; the non-functional and external-interface requirements set the expectations around them, from reliability,

usability, performance, and security to how the apps handle devices and the network. Everything here is written so it can be checked against the finished system later.

2.1 User Classes and Characteristics

The system has three kinds of user, administrator, teacher, and student, and each works in both a mobile and a desktop app, which gives six user classes in all. The same person may use both device types; they are listed separately because the setting, the screen size, and the file and printing actions available differ between a phone and a desktop. Table 2-1 describes each.

Table 2-1: User Classes and Characteristics

User Class	Characteristics
Admin Mobile User	An authorised administrator who needs quick access to summaries, approvals, people, academic records, notices, and reports while moving around the college. Actions are limited to what the account permits.
Admin Desktop User	The same administrator doing longer office work at a desk, managing large lists, timetable and fee work, and report review, with exporting and printing.
Teacher Mobile User	A faculty member using a phone through the teaching day to check classes, mark attendance, enter marks, request a correction to a locked mark, submit exam papers, record results, and read college information.
Teacher Desktop User	The same teacher doing that work at a desk, with more room for class registers, marks grids, schedules, and student lists, plus desktop file and print actions.
Student Mobile User	An enrolled or newly registered student using a phone to link their account and then check their own attendance, timetable, marks, results, fees, datesheets, and notices.
Student Desktop User	The same student on the desktop, using the larger screen to read their records and to download or print the fee challan.

2.2 Requirement Identifying Technique

Because all six applications are interactive and role-based, the requirements were drawn out mainly through use-case modelling: each activity starts with a user goal, runs through an

expected sequence, and ends in a result you can observe and later test. That framing keeps the requirements readable without asking the reader to know how the apps are built. The sections that follow show how it was applied, reviewing the college's existing processes and each role's responsibilities (§2.2.1), checking the ideas against a cross-platform prototype of the six apps (§2.2.2), writing the activities up as fully dressed use cases (§2.2.3), and summarising them as per-app use-case diagrams (§2.2.4).

2.2.1 Process and Stakeholder Review

The requirements grew out of how the college already works. Its existing paper-based procedures, and the records that move through them, were set against what each role, administrator, teacher, and student, is actually responsible for. Four questions drove the review: who creates a given record, who has to approve it, who may see it, and what should happen when information is missing or wrong. That last group is what later shaped the system's access rules and its input checks. The review split cleanly by role:

- **Administrative work**, college structure, people management, approvals, fees, notices, reporting, and control over who is allowed to change what.
- **Teaching work**, schedules, attendance, marks and locked-mark corrections, examination papers, results, student lists, and notices.
- **Student needs**, registration and account linking, then a personal view of academic records, fee information, datesheets, and notifications.

2.2.2 Prototype and Cross-Platform Review

The screens were reviewed as working prototypes (collected in Appendix C) to check that every activity had a clear way in, understandable feedback, and a layout that made sense on both a phone and a desktop. The same pass surfaced the desktop-specific needs a phone does not have, choosing, opening, and saving files, exporting, and printing. Throughout, a role's mobile and desktop apps were treated as two ways into the same work, not two separate stores of college information: the desktop apps run on the same shared logic as their mobile counterparts, so a requirement satisfied on one platform is satisfied on the other rather than specified twice.

2.2.3 Use Case Descriptions

The use cases below cover the principal interactions across the six applications, written in the fully dressed format. Each one names the responsible actor, the applications that support it, the requirements it traces to, the normal sequence, the exceptions worth planning for, and the rules that keep college records trustworthy. The same records appear in Appendix A; the two are kept identical.

UC-01: Sign In and Open the Correct Application Area (*Table 2-2*)

Field	Detail
Requirement	FR-1
Purpose	Let a registered user sign in and land in the area

Field	Detail
	that matches their role.
Priority	High
Applications	All six
Actors	Administrator, Teacher, Student
Description	A registered user signs in and is taken to the correct role-based home. A still-valid session is reused when the app reopens.
Trigger	The user opens an app or submits the sign-in form.
Preconditions	The user has a registered account; admin and teacher accounts are active.
Postconditions	The right home area is shown and only that role's actions are available.
Normal Flow	1. The user opens the app. 2. It checks for a valid session. 3. If needed, the user enters email and password. 4. The account and role are verified. 5. The role's home opens.
Exceptions	Wrong details give a clear message; a disabled account is refused; a role mismatch routes the user to the right app; a connection failure shows a short retry note without exposing service addresses.
Business Rules	A user reaches only the role and records their account is assigned.
Includes / Extends	Included by every protected use case.

UC-02: Manage Administrator Accounts *(Table 2-3)*

Field	Detail
Requirement	FR-2
Purpose	Let an existing administrator add and maintain other authorised administrators.
Priority	High
Applications	Admin Mobile, Admin Desktop
Actors	Administrator

Field	Detail
Description	An administrator views the administrator directory, creates an account, and maintains its college identity and status.
Trigger	The administrator opens People to Administrators.
Preconditions	Signed in as an active administrator.
Postconditions	The directory reflects the addition or update.
Normal Flow	1. Open the directory. 2. Existing administrators are listed. 3. Select Add Administrator. 4. Enter identity and contact details. 5. The system validates and creates the account. 6. The directory refreshes.
Exceptions	Missing, invalid, or duplicate details are flagged by the field; the last usable administrative access cannot be removed by accident; a service failure leaves the directory unchanged.
Business Rules	Only an administrator may create or maintain administrator accounts; email and identity are unique.
Includes / Extends	Includes UC-01.

UC-03: Manage Academic Structure *(Table 2-4)*

Field	Detail
Requirement	FR-3, FR-4, FR-5
Purpose	Maintain departments, academic sessions, semesters, and subjects, the foundation every academic record hangs off.
Priority	High
Applications	Admin Mobile, Admin Desktop
Actors	Administrator
Description	The administrator creates and reviews departments, sessions, semester progress, and the subjects taught in each semester.
Trigger	The administrator opens Academics.

Field	Detail
Preconditions	Signed in. A department must exist before a session, and a session before its subjects.
Postconditions	The structure is available to timetable, attendance, marks, fee, and reporting work.
Normal Flow	1. Select or create a department. 2. Create or open a session. 3. Review semester and session detail. 4. Add subjects to the right semester. 5. The saved structure is confirmed.
Exceptions	Duplicate identifiers, invalid years, missing names, or incomplete semester detail are rejected with field-level guidance; deleting something other records depend on is blocked or confirmed.
Business Rules	Every session belongs to one department; every subject belongs to a defined session and semester.
Includes / Extends	Includes UC-01.

UC-04: Manage Teachers and Students (Table 2-5)

Field	Detail
Requirement	FR-6, FR-7
Purpose	Maintain the people tied to each academic session and the responsibilities granted to teachers.
Priority	High
Applications	Admin Mobile, Admin Desktop
Actors	Administrator
Description	The administrator adds or imports students, reviews student profiles, creates teacher accounts, and manages teacher status and permissions.
Trigger	The administrator opens People, a session roster, or a student profile.
Preconditions	Signed in; the relevant department and session exist.

Field	Detail
Postconditions	Valid teacher and student records are available to authorised workflows.
Normal Flow	1. Open Teachers or a session's Students. 2. Existing records show. 3. Add, import, or update the person. 4. Identity and session information are validated. 5. The list refreshes with the saved record.
Exceptions	Duplicate roll numbers or emails are rejected; invalid import rows are reported without hiding the good ones; disabling a teacher is confirmed and keeps their history.
Business Rules	A roll number is unique within its session; teacher permissions are granted only by an administrator.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-05: Prepare and Maintain Timetables (*Table 2-6*)

Field	Detail
Requirement	FR-8
Purpose	Build a weekly schedule without booking the same teacher or room into overlapping periods.
Priority	High
Applications	Admin Mobile, Admin Desktop
Actors	Administrator, authorised Teacher
Description	An authorised user creates or updates class periods for sessions, subjects, teachers, rooms, days, and times.
Trigger	The user opens the Master Timetable or a session timetable.
Preconditions	Relevant sessions, subjects, teachers, and rooms exist; the user has timetable permission.
Postconditions	The timetable becomes available to the affected administrators, teachers, and students.
Normal Flow	1. Open the timetable. 2. Pick a day and period.

Field	Detail
	3. Enter session, subject, teacher, room, and time. 4. The system checks for clashes. 5. The period is saved and the timetable refreshes.
Exceptions	A teacher, room, or session clash blocks the save and names the conflict; incomplete times or missing assignments are rejected; cancelling changes nothing.
Business Rules	A teacher, room, or session cannot hold two overlapping periods (enforced by the database's own no-double-booking constraint).
Includes / Extends	Includes UC-01.

UC-06: Review Approval Requests (Table 2-7)

Field	Detail
Requirement	FR-14, FR-28
Purpose	Give authorised staff a controlled way to approve student account links and corrections to locked marks.
Priority	High
Applications	Admin Mobile, Admin Desktop; Teacher Mobile/Desktop for link requests when permitted
Actors	Administrator, authorised Teacher
Description	The reviewer opens a pending request, compares it with college records, and approves or rejects it with a recorded outcome.
Trigger	The reviewer opens a pending request from People or Requests.
Preconditions	A pending request exists and the reviewer has permission.
Postconditions	The request is approved or rejected; an approval is applied to the student link or mark.
Normal Flow	1. Open the request queue. 2. Select a pending request. 3. Compare submitted and existing information. 4. Choose Approve or Reject. 5.

Field	Detail
	Confirm; the decision is recorded.
Exceptions	The reviewer may leave without deciding; a decided request cannot be decided again; a rejection reason is required where asked; a failure leaves the request pending.
Business Rules	Whoever submitted a mark-correction request cannot approve it; each request gets exactly one final decision.
Includes / Extends	Includes UC-01; may extend UC-11 or UC-14.

UC-07: Publish College Information (*Table 2-8*)

Field	Detail
Requirement	FR-18, FR-20, FR-22, FR-24, FR-29
Purpose	Create and publish datesheets, fees, fines, calendar events, and notifications for the intended users.
Priority	High
Applications	Admin Mobile, Admin Desktop; permitted Teacher apps for selected notices and datesheets
Actors	Administrator, authorised Teacher
Description	An authorised user prepares college information, chooses its audience or academic scope, reviews it, and publishes it to the relevant apps.
Trigger	The user selects an Add, Create, Issue, or Publish action.
Preconditions	Signed in with the required permission; any referenced session or student exists.
Postconditions	The item is stored and becomes visible to its intended users according to its publication status and audience.
Normal Flow	1. Open the relevant records area. 2. Enter the content and dates. 3. Choose audience and academic scope. 4. The system validates. 5. The user confirms publication. 6. The published item

Field	Detail
	appears in the list.
Exceptions	Missing dates, invalid amounts, or incomplete audience information prevent publication; the user may save or cancel where allowed.
Business Rules	Only authorised users publish; students see only what is addressed to them or their group; a fine must name a student and amount.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-08: Review Reports and Insights (Table 2-9)

Field	Detail
Requirement	FR-10, FR-11, FR-12, FR-15, FR-16, FR-31
Purpose	Help authorised staff review attendance, academic progress, and college summaries without stitching registers together by hand.
Priority	Medium
Applications	Admin Mobile, Admin Desktop, Teacher Mobile, Teacher Desktop
Actors	Administrator, Teacher
Description	The user opens a report or insight, applies the available academic filters, and reads totals, trends, or students needing attention, within their own authority.
Trigger	The user opens Attendance Records, Insights, or an academic summary.
Preconditions	Signed in; relevant academic records exist.
Postconditions	The report is displayed; viewing changes no records.
Normal Flow	1. Open a reporting area. 2. Available scope and filters show. 3. Pick a department, session, semester, subject, or period where relevant. 4. The authorised summary is prepared. 5. The user reviews or exports it.
Exceptions	No matching records gives an informative

Field	Detail
	empty state; invalid filters are ignored or corrected; a loading failure offers retry.
Business Rules	Administrators see college-wide information; teachers see only their assigned records, the same scoping the database enforces.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-09: Mark Class Attendance *(Table 2-10)*

Field	Detail
Requirement	FR-10
Purpose	Record one class meeting's attendance accurately for every enrolled student.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher picks an assigned class and date, marks each student, checks the totals, and submits the register.
Trigger	The teacher selects Mark Attendance.
Preconditions	Signed in, assigned to the class, roster available.
Postconditions	One attendance record is stored per student and becomes visible to authorised users.
Normal Flow	1. Select an assigned class. 2. The date and roster show. 3. Mark each student present, absent, late, or on leave. 4. Review the totals. 5. Submit; confirmation shows.
Exceptions	An incomplete register is caught before submission; a duplicate register opens for review rather than silently duplicating; a connection failure keeps the screen and offers retry.
Business Rules	A teacher marks only assigned classes; one final status per student per meeting. The admin can read attendance but never writes it.

Field	Detail
Includes / Extends	Includes UC-01.

UC-10: Enter Assessment Marks (Table 2-11)

Field	Detail
Requirement	FR-12
Purpose	Record valid marks for students in an assigned subject and assessment.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher picks a class and assessment, enters each student's mark, clears any validation messages, and submits the list.
Trigger	The teacher selects Marks Entry.
Preconditions	Assigned to the subject; roster and the allowed maximum exist.
Postconditions	Valid marks are stored and become read-only to the teacher unless a correction is approved.
Normal Flow	1. Select session, subject, semester, and assessment type. 2. The roster shows. 3. Enter marks. 4. The system checks values and completeness. 5. Submit. 6. Confirmation and locked status show.
Exceptions	Negative marks, values above the maximum, and missing entries are rejected; an already-submitted list opens locked; a failed submission keeps the entries for retry.
Business Rules	Marks stay within the permitted range (midterm out of 25, sessional out of 15); a submitted mark cannot be changed directly.
Includes / Extends	Includes UC-01; may be extended by UC-11.

UC-11: Request a Mark Correction (Table 2-12)

Field	Detail
-------	--------

Field	Detail
Requirement	FR-13
Purpose	Let a teacher request a justified correction instead of editing a locked mark directly.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher selects a locked mark, enters the proposed value and a reason, and submits it for administrative review.
Trigger	The teacher selects the correction action beside a locked mark.
Preconditions	The teacher owns or is responsible for the mark, and it is locked.
Postconditions	A pending request reaches an administrator while the original mark stays unchanged.
Normal Flow	1. Open the locked award list. 2. Select a student's mark. 3. Enter the proposed mark and reason. 4. The system validates the range and checks for another pending request. 5. Submit; pending status shows.
Exceptions	An invalid proposed mark or missing reason blocks submission; a second pending request for the same mark is refused; cancelling changes nothing.
Business Rules	Only an authorised reviewer applies the change; the original value stays active until approval.
Includes / Extends	Extends UC-10; may lead to UC-06.

UC-12: Submit Examination Paper and Semester Results *(Table 2-13)*

Field	Detail
Requirement	FR-15, FR-17
Purpose	Let a teacher submit required examination material and record final semester outcomes for assigned students.

Field	Detail
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher sets the academic context, uploads an examination paper or records semester results (GPA/CGPA and supply subjects), and submits.
Trigger	The teacher opens Submit Exam Paper or Semester Results.
Preconditions	Assigned to the session or subject; the required students and semester information exist.
Postconditions	The paper or result is stored and available to authorised users according to its status.
Normal Flow	1. Select the session and semester. 2. For a paper, pick the subject and file; for results, enter each student's result. 3. The system validates completeness. 4. Review and submit. 5. Confirmation shows.
Exceptions	Unsupported or missing files, incomplete results, and invalid academic values block submission; cancelling changes nothing; a transfer failure offers retry.
Business Rules	Teachers submit only for assigned work; result values follow the college's grading rules.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-13: View Teaching Schedule and Assigned Students *(Table 2-14)*

Field	Detail
Requirement	FR-6, FR-9
Purpose	Give a teacher a reliable view of assigned classes, periods, and students.
Priority	High
Applications	Teacher Mobile, Teacher Desktop

Field	Detail
Actors	Teacher
Description	The teacher reviews the weekly schedule, the next class, assigned sessions, and the student rosters the timetable permits.
Trigger	The teacher opens Home, Schedule, or My Students.
Preconditions	Signed in with at least one assignment.
Postconditions	The schedule or roster is shown without change.
Normal Flow	1. Open the area. 2. The teacher's assignments are identified. 3. Classes or students are grouped by schedule and session. 4. Select an item. 5. Details show.
Exceptions	No assignment gives a clear empty state; missing information offers refresh; the teacher cannot open an unrelated roster.
Business Rules	Teachers see only students and schedules tied to their assigned work.
Includes / Extends	Includes UC-01.

UC-14: Register and Link a Student Account (*Table 2-15*)

Field	Detail
Requirement	FR-27, FR-28
Purpose	Connect a student's sign-in account to the correct college roll-number record, after review.
Priority	High
Applications	Student Mobile, Student Desktop; reviewed in Admin apps and permitted Teacher apps
Actors	Student, Administrator, authorised Teacher
Description	A student registers, enters identifying college information, submits a link request, and waits for a decision before their records open.
Trigger	A new or unlinked student selects Register or Link College Record.
Preconditions	A valid email and an existing college enrolment

Field	Detail
	record.
Postconditions	A pending request is created; on approval the account links to one student record, on rejection the reason or status shows.
Normal Flow	1. The student registers or signs in. 2. The account is recognised as unlinked. 3. The student enters roll number and identity details. 4. The form is validated and a request submitted. 5. A reviewer verifies it via UC-06. 6. The student sees the outcome and gains access on approval.
Exceptions	Invalid details, an unknown roll number, or an already-linked record block submission; a duplicate pending request is not created; rejected requests can be corrected and resubmitted where allowed.
Business Rules	One account links to one student record, and one student record has one approved account.
Includes / Extends	Includes UC-01 and UC-06.

UC-15: View Personal Attendance and Timetable *(Table 2-16)*

Field	Detail
Requirement	FR-9, FR-11
Purpose	Give a linked student a current view of their class schedule and attendance history.
Priority	High
Applications	Student Mobile, Student Desktop
Actors	Student
Description	The student reviews the weekly timetable, next class, attendance totals, per-subject percentages, and available history for their linked record.
Trigger	The student opens Attendance or Timetable.
Preconditions	Signed in and linked to a college record.
Postconditions	The personal schedule or attendance is shown without change.

Field	Detail
Normal Flow	1. Open the area. 2. The linked session and roll number are identified. 3. Timetable or attendance loads. 4. Select a day, subject, or summary. 5. Details show.
Exceptions	No published timetable or attendance gives an informative empty state; a connection failure offers retry; the student cannot select another student's record.
Business Rules	A student sees only their own attendance and their linked session's timetable.
Includes / Extends	Includes UC-01.

UC-16: View Marks, Results, and Datesheets *(Table 2-17)*

Field	Detail
Requirement	FR-16, FR-19
Purpose	Let a linked student review their assessment information and the published examination schedules.
Priority	High
Applications	Student Mobile, Student Desktop
Actors	Student
Description	The student views subject marks, semester results and progression, and datesheets published for their linked session.
Trigger	The student opens Exams, Marks, Results, or Datesheets.
Preconditions	Signed in and linked; the relevant records exist or are published.
Postconditions	The academic information is shown without allowing student changes.
Normal Flow	1. Open Exams. 2. Select Marks, Results, or Datesheets. 3. Information loads for the linked record and session. 4. Available detail shows. 5. The student reviews it.
Exceptions	Unpublished or unavailable information gives a

Field	Detail
	clear message; incomplete history shows only where valid; a loading failure offers retry.
Business Rules	A student sees only personal marks and results, and only datesheets published for the relevant audience.
Includes / Extends	Includes UC-01.

UC-17: View Fee Challan and Fines (Table 2-18)

Field	Detail
Requirement	FR-21, FR-23
Purpose	Give a student current fee information without a routine trip to the accounts office.
Priority	High
Applications	Student Mobile, Student Desktop
Actors	Student
Description	The student opens the fee area to review the session fee structure, their challan, due dates, the payment note, and any recorded fines.
Trigger	The student selects Fee Challan or Fines.
Preconditions	Signed in and linked; a fee structure or fine record exists where applicable.
Postconditions	The fee and fine information is shown without recording any payment.
Normal Flow	1. Open the fee area. 2. The linked session and roll number are identified. 3. Fee heads, totals, due information, and fines are prepared. 4. The student reviews them. 5. The challan can be opened, downloaded, or printed as a PDF.
Exceptions	No fee structure or fines gives an informative empty state; missing information is never replaced with an invented amount; a loading failure offers retry.
Business Rules	A student sees only their own fee and fine records; showing a challan does not mark it paid.

Field	Detail
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-18: View Notices, Events, and Datesheets *(Table 2-19)*

Field	Detail
Requirement	FR-24, FR-30
Purpose	Give users one place to read college announcements, calendar events, and published datesheets meant for them.
Priority	Medium
Applications	All six
Actors	Administrator, Teacher, Student
Description	The user opens notifications, the calendar, or datesheets and reads the items addressed to their role or academic group.
Trigger	The user selects a notice, event, or datesheet from the relevant area or a notification entry.
Preconditions	Signed in; the item has been published for the user's audience.
Postconditions	The item is shown and may be marked read where supported; the published content is unchanged.
Normal Flow	1. Open the relevant area. 2. Authorised published items are listed. 3. Select an item. 4. Its detail is displayed. 5. The user returns to the list.
Exceptions	An expired, removed, or unavailable item gives a safe message; empty lists explain that nothing has been published.
Business Rules	Only items addressed to the user's role, session, or the general college audience are shown.
Includes / Extends	Includes UC-01.

UC-19: Use Desktop File, Export, and Print Actions *(Table 2-20)*

Field	Detail
Requirement	FR-33
Purpose	Let desktop users complete role-appropriate file, export, and print actions through the familiar operating-system services.
Priority	Medium
Applications	Admin Desktop, Teacher Desktop, Student Desktop
Actors	Administrator, Teacher, Student
Description	A desktop user chooses a file, picks a save location, opens a file, prints available information, or shares from an existing workflow (for example, exporting a report or printing a challan).
Trigger	The user selects Upload, Open, Save, Export, Print, or Share where available.
Preconditions	Signed in, authorised for the underlying record, using a desktop app.
Postconditions	The chosen operating-system action completes, or returns a clear cancellation or failure to the app.
Normal Flow	1. Open an authorised record or form. 2. Select a desktop action. 3. The app opens the matching system dialog (select, save, open, or print). 4. The user completes the system step. 5. The app confirms or returns to the screen.
Exceptions	The user may cancel without changing data; missing file access, an unsupported file type, or no printer gives a clear message; internal service details are never exposed.
Business Rules	Desktop actions never bypass the user's role or record permissions; native system windows may look different from the app.
Includes / Extends	Extends UC-04, UC-07, UC-08, UC-12, UC-17, or UC-18.

UC-20: View and Maintain Personal Profile (Table 2-21)

Field	Detail
Requirement	FR-32
Purpose	Let each signed-in user review their account identity, update the profile fields they are allowed to, and sign out safely.
Priority	Medium
Applications	All six
Actors	Administrator, Teacher, Student
Description	The user opens their profile, reviews account and college identity, edits allowed contact or profile details, and may sign out.
Trigger	The user selects Profile or Sign Out.
Preconditions	Signed in.
Postconditions	Valid permitted changes are stored, or the session ends and protected information is no longer reachable.
Normal Flow	1. Open Profile. 2. Account and college identity show. 3. Edit an allowed field if needed. 4. The system validates and saves. 5. Return or sign out. 6. Sign-out returns to the authentication screen.
Exceptions	Invalid details are flagged without discarding valid ones; protected identity and role fields stay read-only; a failed save keeps the previous profile; cancelling sign-out keeps the session active.
Business Rules	Users change only fields permitted for their role; role, status, and protected college identity need an authorised administrative process.
Includes / Extends	Includes UC-01.

2.2.4 Use Case Diagrams

Figures 2-1 to 2-6 give the main actions in each of the six applications, one diagram per app. The paired mobile and desktop diagrams overlap on purpose, both versions work on the same approved college records, and the desktop diagrams add the file, export, and print actions (UC-19) on top. Each diagram's actions line up with the use cases described in §2.2.3.

- **Figure 2-1**, Admin Mobile Application use case diagram
- **Figure 2-2**, Admin Desktop Application use case diagram
- **Figure 2-3**, Teacher Mobile Application use case diagram
- **Figure 2-4**, Teacher Desktop Application use case diagram
- **Figure 2-5**, Student Mobile Application use case diagram
- **Figure 2-6**, Student Desktop Application use case diagram

> **Figure updates required (these are images, edit the diagram source, not this text):** > > - **All six diagrams (2-1 to 2-6):** the shared use case previously shown as *"View Notices, Events, and Documents"* is now **"View Notices, Events, and Datesheets."** Remove the **documents** use-case node, the Documents feature (prospectus/rules/report uploads) was removed from the system. > - **Admin Mobile & Admin Desktop (2-1, 2-2):** the *"Publish College Information"* use case must no longer include **documents** among what it publishes. It now covers datesheets, fees, fines, calendar events, and notifications only. > - **Desktop diagrams (2-2, 2-4, 2-6):** if the desktop file-action node is labelled *"document actions,"* relabel it **"file, export, and print actions"** (UC-19). That node refers to operating-system file handling, choosing/opening/saving files, exporting, printing, not the removed Documents feature.

2.3 Functional Requirements

The 33 functional requirements (FR-1 to FR-33) are laid out in the order information moves through the college: access and people, academic structure, daily teaching records, examinations, fees, communication, reporting, and the desktop file actions. Each one names an observable service and gives its source, its purpose, the main rule that governs it, its dependencies, and its priority, the same fields used in the tables that follow. Between them they cover all six applications, and every requirement notes which of them it applies to.

2.3.1 FR-1: Sign In and Role Routing

Table 2-22: Description of FR-1

Field	Detail
Identifier	FR-1
Title	Sign In and Role Routing
Requirement	A registered administrator, teacher, or student shall sign in with their email and password through their own mobile or desktop app. The system shall reuse a still-valid session when the app reopens, send the user to the home area for their role, and show a clear, safe message, without exposing any service address, when the details are wrong, the account is inactive, the role does not match the app, or the connection

Field	Detail
	fails.
Source	All user classes
Rationale	Access is decided per role, so every protected action depends on the user's identity and role being settled at sign-in. Getting that right once is what lets the rest of the system trust who is asking.
Business Rule	A user reaches only the area for their account's active role.
Dependencies	None, every other requirement depends on this one.
Priority	High

2.3.2 FR-2: Manage Administrator Accounts

Table 2-23: Description of FR-2

Field	Detail
Identifier	FR-2
Title	Manage Administrator Accounts
Requirement	An administrator shall be able to see the existing administrators and add or maintain another administrator account from either admin app. The system shall validate the required identity and contact details, refuse a duplicate account, and make sure a failed creation leaves no partial account behind.
Source	College administration
Rationale	The college needs to share administrative work across more than one person without anyone setting up accounts outside the system. Creating them in-app keeps that under an existing administrator's control.
Business Rule	Only an active administrator may grant administrative access; an account's email and identity must be unique.
Dependencies	FR-1

Field	Detail
Priority	High

2.3.3 FR-3: Manage Departments

Table 2-24: Description of FR-3

Field	Detail
Identifier	FR-3
Title	Manage Departments
Requirement	An administrator shall be able to create, view, and update departments from the admin apps. A department with a missing or duplicate identifier shall not be saved, and deleting one shall require confirmation, because removing a department also removes the academic sessions beneath it and everything those sessions hold.
Source	College administration
Rationale	A department sits at the top of the structure: sessions, people, and academic records all hang beneath it. It has to exist before any of them, and be removed with care.
Business Rule	Each department has a unique identifier and a clear display name.
Dependencies	FR-1
Priority	High

2.3.4 FR-4: Manage Academic Sessions

Table 2-25: Description of FR-4

Field	Detail
Identifier	FR-4
Title	Manage Academic Sessions
Requirement	An administrator shall be able to create, view, update, and promote academic sessions within a department. The system shall check the session's year, shift, and semester before saving and reject duplicate or inconsistent details.

Field	Detail
	Promoting a session advances it to the next semester, and graduates it after the eighth, while a completed session can be archived and removed.
Source	College administration
Rationale	Students, curriculum, timetables, fees, and results are all organised by academic session, so the session is the unit the rest of the academic year is built around.
Business Rule	A session belongs to one department and may advance only within its permitted semester range, up to graduation at semester 8.
Dependencies	FR-3
Priority	High

2.3.5 FR-5: Manage Curriculum

Table 2-26: Description of FR-5

Field	Detail
Identifier	FR-5
Title	Manage Curriculum
Requirement	An administrator shall be able to add, view, update, and remove the subjects that make up a session's curriculum, semester by semester. Each subject carries a course code, name, credit hours, and a THEORY or LAB type, and may be marked elective; the system shall validate these details and refuse a duplicate course code within the same session and semester.
Source	College administration and teaching staff
Rationale	Attendance, marks, timetables, and results all attach to a subject, so the curriculum has to be agreed before any of them can be recorded.
Business Rule	Every subject belongs to one session and semester; a course code is unique within that session and semester.

Field	Detail
Dependencies	FR-4
Priority	High

2.3.6 FR-6: Manage Students and Session Rosters

Table 2-27: Description of FR-6

Field	Detail
Identifier	FR-6
Title	Manage Students and Session Rosters
Requirement	An administrator shall be able to add students one at a time or import a whole roster, view each student's card and profile, and update valid details. The system shall report duplicate roll numbers, missing required identity information, and invalid import rows without silently overwriting existing students, and shall keep a roster within its session's capacity.
Source	College administration and teaching staff
Rationale	Attendance, marks, fees, and account linking all depend on an accurate list of the students actually enrolled in a session.
Business Rule	A roll number identifies one student within a session; a student's enrolment status, active, promoted, repeated, withdrawn, or graduated, is carried across terms, and editing profile details never discards academic history.
Dependencies	FR-4
Priority	High

2.3.7 FR-7: Manage Teachers and Permissions

Table 2-28: Description of FR-7

Field	Detail
Identifier	FR-7
Title	Manage Teachers and Permissions

Field	Detail
Requirement	An administrator shall be able to create teacher accounts, view and update permitted teacher details, reset a teacher's password, and disable, ban, or reactivate an account with confirmation. The administrator shall also grant or remove a teacher's delegated permissions, approving student link requests, editing the timetable, sending notifications, and managing datesheets. Invalid or duplicate account details shall not be saved.
Source	College administration
Rationale	Teaching access and any delegated duties stay under administrative control, so a teacher holds only the extra powers an administrator has explicitly granted.
Business Rule	Only an administrator may change a teacher's status or permissions; a status change preserves the teacher's historical teaching records.
Dependencies	FR-1
Priority	High

2.3.8 FR-8: Manage Timetables

Table 2-29: Description of FR-8

Field	Detail
Identifier	FR-8
Title	Manage Timetables
Requirement	An administrator, or a teacher granted timetable permission, shall be able to create, view, update, and remove class periods in the master timetable and in each session timetable. A period sets the day, time, subject, teacher, and room, rooms and buildings are chosen from managed lists, and the timetable also allows zero periods, breaks, and merged lectures. Missing assignments and any overlap of the same teacher, room, or session in the same time slot shall be caught before the period is saved.

Field	Detail
Source	College administration and teaching staff
Rationale	One shared timetable is what keeps rooms, teachers, subjects, and sessions coordinated, and it is what the teacher and student schedule views read from.
Business Rule	The same teacher, room, or session cannot hold two overlapping periods; the database refuses such a save.
Dependencies	FR-4, FR-5, FR-7
Priority	High

2.3.9 FR-9: View Personal Schedule

Table 2-30: Description of FR-9

Field	Detail
Identifier	FR-9
Title	View Personal Schedule
Requirement	A teacher shall be able to view their assigned weekly classes and upcoming periods, and edit the timetable where they have been granted timetable permission; a student shall be able to view the timetable of their linked session. Where nothing has been published, the app shall show an informative empty state rather than unrelated data.
Source	Teaching staff and students
Rationale	Each user needs quick access to the schedule that actually applies to them, the teacher to their own classes, the student to their session.
Business Rule	A teacher sees only their assigned periods; a student sees only the timetable of the session their account is linked to. Both read the same published timetable (FR-8).
Dependencies	FR-8, FR-27
Priority	High

2.3.10 FR-10: Mark Attendance

Table 2-31: Description of FR-10

Field	Detail
Identifier	FR-10
Title	Mark Attendance
Requirement	An assigned teacher shall be able to select one of their classes and a date, mark each enrolled student present, absent, or on leave, flagging a present student as late where needed, review the totals, and submit the register. Incomplete entries, a duplicate register for the same meeting, an unassigned class, and a failed submission shall each be identified clearly.
Source	Teaching staff and college administration
Rationale	A digital register replaces manual counting and makes the same record available to the student and to administrative reports without re-entry.
Business Rule	One final status, present, absent, or leave, is stored per student for each class meeting, with an optional late flag. Only the assigned teacher may write attendance; an administrator can read it for reports but never records or edits it.
Dependencies	FR-6, FR-8
Priority	High

2.3.11 FR-11: View Attendance History

Table 2-32: Description of FR-11

Field	Detail
Identifier	FR-11
Title	View Attendance History
Requirement	A teacher shall be able to review attendance history for their assigned classes, an administrator shall be able to review authorised attendance reports, and a student shall be able to see only their own attendance, a per-subject present/absent/leave breakdown with an

Field	Detail
	attendance percentage they can check against the 75% requirement. Where nothing has been recorded, the app shall show an empty state rather than reporting zero attendance.
Source	All user classes
Rationale	Attendance is only useful if the right person can review it in time, the teacher for their classes, the administration across the college, and the student for their own standing.
Business Rule	What each user sees follows their role, teaching assignment, session, and linked student identity.
Dependencies	FR-10, FR-27
Priority	High

2.3.12 FR-12: Enter Assessment Marks

Table 2-33: Description of FR-12

Field	Detail
Identifier	FR-12
Title	Enter Assessment Marks
Requirement	An assigned teacher shall be able to enter a score for each student in an approved assessment and submit the complete award list. Midterm marks are out of 25 and sessional marks out of 15 (entered as attendance, assignment, and quiz components of 5 each). The system shall reject negative values, values above the assessment's maximum, incomplete entries, and unassigned classes with clear guidance. Once submitted, the marks lock against direct editing.
Source	Teaching staff and college administration
Rationale	Marks have to be entered accurately and then protected from quiet, unrecorded changes.
Business Rule	A score stays within its assessment's maximum, midterm 25, sessional 15, and locks after the first successful submission; changing a locked

Field	Detail
	mark then requires an approved correction (FR-13).
Dependencies	FR-5, FR-6, FR-8
Priority	High

2.3.13 FR-13: Request Mark Correction

Table 2-34: Description of FR-13

Field	Detail
Identifier	FR-13
Title	Request Mark Correction
Requirement	A teacher responsible for a locked mark shall be able to propose a corrected value and give a reason for the change. The system shall reject an out-of-range value, a missing reason, or a second request while one is already pending, and shall leave the original mark in place until the request is decided.
Source	Teaching staff and college administration
Rationale	Genuine mistakes have to be fixable, but a submitted mark should never change quietly, a correction leaves a visible, reviewable trail.
Business Rule	A locked mark changes only after an administrator approves the request; the teacher who raises a correction cannot approve it, and only one request may be pending for a mark at a time.
Dependencies	FR-12
Priority	High

2.3.14 FR-14: Review Mark Correction

Table 2-35: Description of FR-14

Field	Detail
Identifier	FR-14

Field	Detail
Title	Review Mark Correction
Requirement	An administrator shall be able to open a pending mark-correction request, see the old and proposed values side by side, read the teacher's reason, and approve or reject it. A request that has already been decided cannot be decided again, and if the decision fails to save, the original mark and the request's status are both left untouched.
Source	College administration
Rationale	An independent second look at any change to a submitted mark is what keeps the assessment record trustworthy.
Business Rule	The teacher who raised the request cannot approve it; approving records the decision and applies exactly the proposed value, nothing else.
Dependencies	FR-13
Priority	High

2.3.15 FR-15: Record Semester Results

Table 2-36: Description of FR-15

Field	Detail
Identifier	FR-15
Title	Record Semester Results
Requirement	An assigned teacher shall be able to record each student's semester result, GPA, CGPA, result status, class position, any carried supply (retake) subjects, and remarks, for a given semester and term. The system shall flag missing students, out-of-range values, and incomplete entries before the result is saved.
Source	Teaching staff and college administration
Rationale	Students and administrators need one agreed semester-level outcome per student, recorded through a single controlled path so GPA and

Field	Detail
	CGPA stay consistent everywhere they appear.
Business Rule	Results may be recorded only for the teacher's assigned session and semester, using allowed academic values; every result is written through the one controlled result path, which is the only way a teacher can set a GPA.
Dependencies	FR-5, FR-6, FR-8
Priority	High

2.3.16 FR-16: View Marks and Semester Results

Table 2-37: Description of FR-16

Field	Detail
Identifier	FR-16
Title	View Marks and Semester Results
Requirement	A linked student shall be able to view their own subject marks, midterm out of 25, sessional out of 15, and the total, alongside their semester results (GPA, CGPA, any supply subjects, and progression). Administrators and authorised teachers shall be able to review results within their permitted scope. Where a mark or result has not been published, or a student was marked absent, it shall be shown as such rather than as a zero or a completed result.
Source	Students, teaching staff, and college administration
Rationale	Seeing marks and results on a phone removes the wait for manually prepared copies and trips to the office.
Business Rule	A student sees only their own marks and results; a teacher sees the records for their assigned classes; an administrator sees authorised college-wide records.
Dependencies	FR-12, FR-15, FR-27
Priority	High

2.3.17 FR-17: Submit Examination Paper

Table 2-38: Description of FR-17

Field	Detail
Identifier	FR-17
Title	Submit Examination Paper
Requirement	For a subject that appears on a published mid-term datesheet, the assigned teacher shall be able to attach an examination-paper PDF to that datesheet slot and submit it, optionally with a short description. The system shall reject a missing file, an unsupported or oversize file (PDFs up to 5 MB), a slot the teacher is not assigned to, and a transfer failure, each with a clear message. Only one active paper is kept per slot; submitting again replaces it.
Source	Teaching staff and college administration
Rationale	Binding each paper to a published datesheet slot keeps every submission attached to a real, scheduled exam rather than a free-form choice.
Business Rule	A teacher may submit only against their own assigned slots on a published mid-term datesheet; the file is stored in the exam-papers bucket. Administrators can view and download submitted papers but do not review or approve them.
Dependencies	FR-5, FR-8, FR-18
Priority	High

> **Change from the earlier report:** exam-paper submission was reworked (20260910130000_exam_paper_rework.sql). A paper is now tied to a **published datesheet slot** instead of a free (session, subject, exam-type) selection, it applies to **mid-term exams only** (the exam_type field was dropped, since datesheets never cover sessional), and the **admin "review/approval" step was removed** (the review_status/reviewed_by/reviewed_at/teacher_notes columns and their guard trigger were dropped). Any description of an admin approving or reviewing papers is stale and has been removed.

2.3.18 FR-18: Build and Publish Datesheet

Table 2-39: Description of FR-18

Field	Detail
Identifier	FR-18
Title	Build and Publish Datesheet
Requirement	An administrator, or a teacher with the manage-datesheets permission, shall be able to build an examination datesheet for one session and semester, schedule each of that curriculum's subjects with a date, time, and a real building and room, review it, and publish it to that session's students. The system shall block publication when a subject is scheduled twice, when two papers fall on the same date within the datesheet, when a room or invigilator clashes with any other datesheet, or when required entries are missing or invalid.
Source	College administration and teaching staff
Rationale	A published datesheet gives every affected user one approved exam schedule, and, because exam papers attach to its slots, it is also what paper submissions (FR-17) hang off.
Business Rule	A datesheet covers exactly one (session, semester); its papers are the session's own curriculum subjects, each placed in a real room; only an administrator or a permitted teacher may publish; a student sees only the datesheet for their linked session.
Dependencies	FR-4, FR-5, FR-8
Priority	High

> **Change from the earlier report:** the datesheet model was reworked (20260908120000_datesheet_rework.sql). A datesheet is now scoped to one (**session, semester**) instead of a free-floating title; its papers are drawn from the session's **real curriculum subjects** and placed in **real buildings/rooms** (previously free text); the **sessional exam type was dropped** (sessional is continuous assessment, not scheduled); and conflict checks now run **server-side across all datesheets** (a subject can't be scheduled twice, two papers can't share a date within one datesheet, and rooms/invigilators can't clash across datesheets). Any earlier description of a free-title datesheet or free-text rooms is stale.

2.3.19 FR-19: View Datesheet

Table 2-40: Description of FR-19

Field	Detail
Identifier	FR-19
Title	View Datesheet
Requirement	An administrator or teacher shall be able to review the datesheets within their authority, and a student shall be able to view the datesheet published for their linked session and semester, each paper listed with its subject, date, time, and room. A draft, unrelated, removed, or unavailable datesheet shall not appear as an active schedule, and where none has been published the student shall see a clear empty state.
Source	All user classes
Rationale	An exam schedule should reach students on their own device, not only on a physical noticeboard.
Business Rule	A student sees only published datesheets, and only the one for their linked session.
Dependencies	FR-18, FR-27
Priority	High

2.3.20 FR-20: Manage Session Fee Structure

Table 2-41: Description of FR-20

Field	Detail
Identifier	FR-20
Title	Manage Session Fee Structure
Requirement	An administrator shall be able to set a session's fee structure, the fee heads and their amounts, the cadence (annual or per-semester), the due date, a late-fine note, and a payment note, and update it later. The system shall reject negative amounts, missing required fee information, and invalid dates before saving.

Field	Detail
Source	College administration and accounts office
Rationale	A student's challan can only be shown once the session's fee structure has been agreed and entered.
Business Rule	A fee structure belongs to one session and is set only by an administrator; the payment note makes clear that fees are paid at the college accounts office, never through the app or a bank.
Dependencies	FR-4
Priority	High

2.3.21 FR-21: View Fee Challan

Table 2-42: Description of FR-21

Field	Detail
Identifier	FR-21
Title	View Fee Challan
Requirement	A linked student shall be able to view their fee challan, built from their linked session's current fee structure, and generate it as a printable PDF. The challan prints three copies on one page, a Student copy, a College copy, and a Clerk copy, each listing the fee heads and the total. Where no fee structure has been published, the app shall show an informative message; opening, downloading, or printing the challan never records a payment.
Source	Students and accounts office
Rationale	A student should be able to get their challan from their own device instead of making an information-only trip to the accounts office.
Business Rule	A student sees only the fee information for their linked record and session; the app presents the challan but never collects payment.
Dependencies	FR-20, FR-27

Field	Detail
Priority	High

2.3.22 FR-22: Issue Student Fine

Table 2-43: Description of FR-22

Field	Detail
Identifier	FR-22
Title	Issue Student Fine
Requirement	An administrator shall be able to issue a fine to an identified student, recording its category, amount, and reason. The system shall reject a missing student, a non-positive amount, or an empty reason. A fine is informational only, the app records that it was issued, along with who issued it and when, but tracks no paid or unpaid status.
Source	College administration
Rationale	A fine needs a clear, attributable record rather than an informal note passed around on paper.
Business Rule	Only an administrator may issue a fine (teachers can see the fines for their own classes but cannot create them); every fine identifies the issuer, the student, the amount, the category, and the reason.
Dependencies	FR-6
Priority	Medium

2.3.23 FR-23: View Student Fines

Table 2-44: Description of FR-23

Field	Detail
Identifier	FR-23
Title	View Student Fines
Requirement	A linked student shall be able to see their own fines, each with its category, amount, and reason, alongside their fee information, while

Field	Detail
	administrators and authorised teachers may view fines within their permitted scope. Where no fine exists, the app shall say so plainly rather than showing an unexplained zero.
Source	Students, teaching staff, and college administration
Rationale	Everyone involved should be able to see which fines were recorded and why, without chasing a paper note.
Business Rule	A student sees only the fines attached to their linked record; a teacher sees fines for their assigned classes; an administrator sees all. Fines are informational, the app records them but takes no payment.
Dependencies	FR-22, FR-27
Priority	Medium

2.3.24 FR-24: Manage Calendar Events

Table 2-45: Description of FR-24

Field	Detail
Identifier	FR-24
Title	Manage Calendar Events
Requirement	An administrator shall be able to create, update, and remove college calendar events, each with a title, a type, a date or date range, optional start and end times, a venue, a description, and an audience. The system shall reject an invalid date range, a missing title, or incomplete audience information before saving.
Source	College administration
Rationale	Holidays, examinations, deadlines, and events belong on one current college calendar rather than scattered across notices.
Business Rule	An event's audience decides who sees it, everyone, a particular role, a department, or a session; a college-wide event leaves the

Field	Detail
	department and session unset.
Dependencies	FR-1
Priority	Medium

2.3.25 FR-25: Publish College Documents (Withdrawn)

Table 2-46: Description of FR-25

Field	Detail
Identifier	FR-25
Title	Publish College Documents (<i>Withdrawn</i>)
Requirement	Withdrawn. The Documents feature (uploading and publishing prospectus, rules, and report files) was removed from the system, so this requirement no longer applies. No application offers a publish-documents capability.
Source	N/A
Rationale	Document publishing was dropped during development; the public.documents table and its local cache were removed (20260827190000_drop_documents_feature_table.sql; Room MIGRATION_30_31).
Business Rule	N/A
Dependencies	N/A
Priority	Withdrawn

> **Note:** the FR-25 identifier is kept so FR-26...FR-33, the traceability matrix, and every cross-reference stay stable. Fully deleting this entry and renumbering the later requirements is an alternative the team may choose instead.

2.3.26 FR-26: View and Download Documents (Withdrawn)

Table 2-47: Description of FR-26

Field	Detail
Identifier	FR-26
Title	View and Download Documents (<i>Withdrawn</i>)

Field	Detail
Requirement	Withdrawn. The Documents feature (viewing and downloading published prospectus, rules, and report files) was removed from the system, so this requirement no longer applies. No application offers a view- or download-documents capability, and UC-18 has been renamed "View Notices, Events, and Datesheets" to match.
Source	N/A
Rationale	Document handling was dropped during development; the public.documents table and its local cache were removed (20260827190000_drop_documents_feature_table.sql; Room MIGRATION_30_31).
Business Rule	N/A
Dependencies	N/A
Priority	Withdrawn

> **Note:** the FR-26 identifier is kept so FR-27...FR-33, the traceability matrix, and every cross-reference stay stable. Fully deleting this entry and renumbering the later requirements is an alternative the team may choose instead.

2.3.27 FR-27: Submit Student Link Request

Table 2-48: Description of FR-27

Field	Detail
Identifier	FR-27
Title	Submit Student Link Request
Requirement	A registered but unlinked student shall be able to claim their college record by submitting the roll number, chosen from the list of unclaimed roll numbers for their session, together with identity details (CNIC/B-form, name, and date of birth) for an authorised reviewer to check. The system shall refuse an unknown record, a mismatch, a second request while one is already pending, or a record that is already linked. The rest of the app stays gated until the request is approved.

Field	Detail
Source	Students and college administration
Rationale	Self-registration must never hand a student access to someone else's academic record, so a claim is checked against the college roster before anything opens.
Business Rule	One account links to one student record, and each student record has at most one approved account; a request moves through pending, approved, or rejected.
Dependencies	FR-1, FR-6
Priority	High

2.3.28 FR-28: Review Student Link Request

Table 2-49: Description of FR-28

Field	Detail
Identifier	FR-28
Title	Review Student Link Request
Requirement	An administrator, or a teacher granted the approve-link-requests permission, shall be able to compare a pending link request against the college roster and approve or reject it. A decided request cannot be processed again, and a failed decision leaves it pending. An administrator may also revoke or re-link a record later; doing so downgrades the previous account's now-stale approval so a record is never left with two active links.
Source	College administration and teaching staff
Rationale	An independent check keeps a student record from being linked to the wrong account, and keeps exactly one account attached to it over time.
Business Rule	Approval links one account to one matching student record; rejection records the decision without linking; a record has at most one live approved link, and re-linking or delinking clears

Field	Detail
	the earlier one.
Dependencies	FR-27
Priority	High

2.3.29 FR-29: Publish Notifications

Table 2-50: Description of FR-29

Field	Detail
Identifier	FR-29
Title	Publish Notifications
Requirement	An administrator, or a teacher granted the send-notifications permission, shall be able to compose a notification with a title and message, choose its audience, and send it. The audience is a role (or everyone) and can be narrowed to a department or a session. Empty messages, a missing title, and an invalid audience shall be rejected.
Source	College administration and teaching staff
Rationale	Announcements need to reach the right people promptly instead of relying on a noticeboard.
Business Rule	Only an administrator or a permitted teacher may send a notification, and every notification names an audience. Delivery is in-app, there is no external push.
Dependencies	FR-1
Priority	Medium

2.3.30 FR-30: View Notifications

Table 2-51: Description of FR-30

Field	Detail
Identifier	FR-30
Title	View Notifications
Requirement	A signed-in user shall be able to see the

Field	Detail
	notifications addressed to their role or academic group, open one for its full detail, and see a badge with their unread count. Notifications meant for other audiences shall not appear, and a loading failure shall offer a retry.
Source	All user classes
Rationale	Everyone needs one consistent place to find current college alerts rather than checking a noticeboard.
Business Rule	What a user sees follows each notification's target audience; the read/unread state is kept per viewer, so opening a notification clears it for that user alone and does not change it for anyone else.
Dependencies	FR-29
Priority	Medium

2.3.31 FR-31: View Insights and Reports

Table 2-52: Description of FR-31

Field	Detail
Identifier	FR-31
Title	View Insights and Reports
Requirement	An administrator shall be able to view college-wide summaries, session overviews, students at academic risk, and per-exam statistics, while a teacher shall be able to view the same kinds of summary limited to their assigned classes. Filters shall cover department, session, semester, subject, or period where they apply, and no-data and loading states shall be explained clearly.
Source	College administration and teaching staff
Rationale	Ready-made summaries let staff spot attendance and performance problems without compiling registers by hand.

Field	Detail
Business Rule	The scope of every summary follows the signed-in user's role and assignments, enforced by the database's row-level rules on the underlying records rather than by extra logic in the app.
Dependencies	FR-4, FR-6, FR-10, FR-12, FR-15
Priority	Medium

> **Change from the earlier report:** four SQL insight views (at_risk_students, session_overview, session_attendance_summary, exam_stats) were dropped as unused (20260905000005_drop_unused_insights_views.sql). The app now computes the same summaries from the base tables and caches them locally; the admin/teacher tiering comes from Row-Level Security on those base tables, so each user's own reads are already scoped. (Note: the Executive Summary's phrase "built directly on Postgres views" is therefore no longer precise, the tiering is still RLS-driven, but the figures are computed over base tables, not read from dedicated views. Worth adjusting T-FM1 if strict accuracy is wanted.)

2.3.32 FR-32: View and Maintain Personal Profile

Table 2-53: Description of FR-32

Field	Detail
Identifier	FR-32
Title	View and Maintain Personal Profile
Requirement	A signed-in user shall be able to view their account and college profile, the parts their role is allowed to see, update the fields they may edit, and sign out. An invalid edit shall not overwrite valid information, and protected fields (role, account status, and the email that serves as the account's identity) shall stay read-only, changing only through an authorised administrative process.
Source	All user classes
Rationale	A user needs to confirm their own identity and keep their contact details current, without being able to alter the fields that decide who they are in the system.
Business Rule	A user edits only the profile fields permitted for their role; email, role, and status are

Field	Detail
	administrator-managed.
Dependencies	FR-1
Priority	Medium

2.3.33 FR-33: Use Desktop File and Print Services

Table 2-54: Description of FR-33

Field	Detail
Identifier	FR-33
Title	Use Desktop File and Print Services
Requirement	The Admin, Teacher, and Student desktop apps shall offer role-appropriate file services through the operating system's own dialogs, choosing a file to open, picking a save location, exporting a report to CSV or PDF, printing a challan or record, and sharing where supported. Cancellation, a missing application or printer, insufficient permission, or an unsupported file shall return a clear result without changing college data.
Source	Desktop user classes
Rationale	At a desk, users expect these file, export, and print steps to run through the familiar operating-system dialogs rather than a bespoke in-app version.
Business Rule	A desktop file action never bypasses the app's access rules; the native operating-system windows sit outside the app's own visual design.
Dependencies	FR-1, plus the requirement that owns the record being exported or printed
Priority	Medium

2.4 Non-Functional Requirements

These quality requirements apply across all six applications and cover four areas: how reliably the system holds and syncs data (reliability), how easily people can work in it (usability), how quickly it responds (performance), and how it protects records and access (security). Each is

written as an observable target, so it can be checked during testing rather than left as a general intention.

2.4.1 Reliability

The system is offline-first: each app keeps a local cache and syncs only the records that have changed, so most of the reliability requirements are about never losing what a user has already seen or entered.

Table 2-55: Reliability Requirements

ID	Requirement	Applications	Priority
NFR-R1	Once the system confirms a record is saved, it stays available after the app is closed and reopened, and is shown from the local cache even with no connection.	All six	High
NFR-R2	A temporary loading or connection failure never closes the app or clears what is already on screen; cached data stays visible and the user is offered a clear retry.	All six	High
NFR-R3	On an administrator's first entry in a session, the core reference data, departments, teachers, students, sessions, requests, and summaries, begins loading automatically, without a screen-by-screen refresh.	Admin Mobile, Admin Desktop	High
NFR-R4	A valid session is restored when the app reopens; an expired or signed-out session returns the user to sign-in without showing any	All six	High

ID	Requirement	Applications	Priority
	protected data.		
NFR-R5	Every create, update, upload, approval, or delete is validated on the server; a failed action leaves the previous record unchanged unless the system confirms success.	All six	High

2.4.2 Usability

All six apps are built from one shared component library, the "Modernist" design system on Material 3, which is what keeps navigation, wording, and behaviour consistent across roles and platforms.

Table 2-56: Usability Requirements

ID	Requirement	Applications	Priority
NFR-U1	A user can reach any primary destination for their role within three selections from the role's home.	All six	High
NFR-U2	Required fields, invalid values, and unavailable actions are explained in plain language beside the form or control, before the user is asked to redo the whole task.	All six	High
NFR-U3	A destructive or high-impact action names the affected record and asks for confirmation first.	Admin and Teacher apps	High
NFR-U4	On mobile, controls for primary actions give a touch target of at least 48 x 48 dp wherever the	Three mobile apps	Medium

ID	Requirement	Applications	Priority
	screen allows.		
NFR-U5	The mobile and desktop version of a role use the same labels, the same order of major destinations, the same status meanings, and the same form rules.	All six	High
NFR-U6	Every main screen has a clear title, predictable back behaviour, a way to refresh, loading feedback, and an understandable empty or error state.	All six	High
NFR-U7	Body text meets at least 4.5:1 contrast, icon-only controls carry content descriptions, status is never conveyed by colour alone, the layout respects the system font scale, and a dark theme is available.	All six	Medium

2.4.3 Performance

Performance is stated as targets for a single-college data volume on a stable connection. The measured local baseline is reported in Chapter 5 (§5.4); behaviour under live-network delay and concurrent use is left to acceptance testing rather than asserted here.

Table 2-57: Performance Requirements

ID	Requirement	Applications	Priority
NFR-P1	On a stable connection, the first load of a normal list or summary completes within five seconds at single-college data volumes.	All six	High

ID	Requirement	Applications	Priority
NFR-P2	A screen that already holds valid cached data shows it within two seconds while it checks for updates in the background.	All six	Medium
NFR-P3	After a teacher selects a class, its attendance roster is ready to mark within three seconds under stable conditions.	Teacher Mobile, Teacher Desktop	High
NFR-P4	The admin start-up data load begins as soon as the role is resolved and never blocks the dashboard from showing progress or the summaries already available.	Admin Mobile, Admin Desktop	High
NFR-P5	Any operation taking longer than one second shows visible progress and blocks accidental duplicate submission while it runs.	All six	High

2.4.4 Security

The system's access rules live in the database, not in the apps: Row-Level Security on every table decides what each signed-in user may read and write, GoTrue handles authentication, and the few service-role operations run through edge functions that first verify the caller is an administrator. Connection secrets stay out of the repository. The requirements below are stated so each can be checked.

Table 2-58: Security Requirements

ID	Requirement	Applications	Priority
NFR-S1	A user can see and do only what their authenticated role and current account status	All six	High

ID	Requirement	Applications	Priority
	allow.		
NFR-S2	A student sees only the records linked to their approved college identity, plus information published to an audience that includes them.	Student Mobile, Student Desktop	High
NFR-S3	A teacher sees or changes only the students, classes, marks, attendance, and delegated duties tied to their assigned work and granted permissions.	Teacher Mobile, Teacher Desktop	High
NFR-S4	Error messages, user-visible logs, and support text never reveal passwords, access tokens, personal secrets, database addresses, or full service-request URLs.	All six	High
NFR-S5	After sign-out, protected screens and personal records are not reachable through back navigation or by reopening the app.	All six	High
NFR-S6	Approval decisions and other sensitive changes keep a record of the responsible account, the action taken, and the time, for later review.	Admin and authorised Teacher apps	High
NFR-S7	Sensitive write paths are constrained in the database itself: an administrator can read but never write	All six	High

ID	Requirement	Applications	Priority
	attendance, a teacher records a GPA only through the controlled semester-result path, and a locked mark changes only through an approved correction.		

2.5 External Interface Requirements

This section sets out how the six applications meet the world around them across four fronts: the **user interface** each role works through, the **software** the apps depend on (the Supabase backend and the libraries behind it), the **hardware** they run on (Android phones and desktop computers), and the **communications** that carry data between the apps and the backend. The detailed screen design itself is covered separately in Chapter 3.

2.5.1 User Interface Requirements

All six apps render through one shared Compose UI kit built on Material 3, the "Modernist" theme, with a signature accent per app (crimson for admin, gold for teacher, emerald for student). Every screen follows the same patterns: a titled header, five-tab bottom navigation on the mobile apps, data tables for structured records, and consistent loading, empty, error, and offline states, in both light and dark themes.

Table 2-59: User Interface Requirements

Application	Required interface characteristics
Admin Mobile	Portrait-first dashboard with five bottom-nav areas; compact cards for college totals and records; short, touch-friendly forms and confirmation dialogs.
Admin Desktop	A wider administrative workspace using card, list, and table layouts, with file, export, and print actions, without changing the role's access rules.
Teacher Mobile	Fast access to today's schedule, attendance, exams, marks, and menu actions; class and student controls sized for use during a lesson.
Teacher Desktop	Wider attendance, marks, schedule, and student views that keep the same labels, status meanings, validation, and navigation order as the mobile app.

Application	Required interface characteristics
Student Mobile	A simple personal home and five main areas, attendance, exams, timetable, and more; forms limited to registration, account linking, and permitted profile edits.
Student Desktop	Readable personal academic, fee, and notice views with open, save, and print actions; no access beyond what the student mobile app allows.
Common	Every main screen uses the shared title treatment, clear back behaviour, refresh, a notification entry where relevant, consistent status colours, and readable typography, with plain loading, empty, validation, and error messages, in light and dark themes, with touch targets of at least 48 dp and content descriptions on icon-only controls.

2.5.2 Software Interfaces

The apps interface with a Supabase backend and a set of Kotlin and Compose libraries. Each is listed below with the version in use and what the apps rely on it for.

Table 2-60: Software Interfaces

Software interface	Version / platform	Required interaction
Supabase Kotlin SDK	3.1.4	The client all six apps use to reach the backend: PostgREST for college records, GoTrue for sign-in, Storage for exam-paper PDFs and profile photos, and the admin-only edge functions.
PostgreSQL (via Supabase)	Supabase-managed	Stores all administrative, academic, fee, approval, communication, and reporting records; Row-Level Security governs who may read and write each row.
Ktor	3.1.1	The HTTP engine the Supabase client runs on.
Room	2.6.1	The on-device database that

Software interface	Version / platform	Required interaction
		caches records so the mobile apps work offline and sync when back online.
Hilt	2.52	Dependency injection that wires repositories and view models in the mobile apps.
Kotlin · Jetpack Compose · Compose Multiplatform	2.1.0 · BOM 2024.12.01 · 1.7.3	The language and UI toolkits the apps are built with (Compose for the Android apps, Compose Multiplatform for the desktop apps).
Android operating system	Android 7.0 (API 24) or later	Runs the three mobile apps and provides device storage, the app lifecycle, and secure internet access.
Windows operating system	Windows 10 or later	Runs the three desktop apps and provides native file selection, save location, file opening, printing, and sharing.
Email delivery (via GoTrue)	Configured in the Supabase project	Delivers account-verification and password-recovery messages, without needing a separate college email application.

2.5.3 Hardware Interfaces

The apps need no specialised hardware, ordinary Android phones and desktop computers, with a printer only where a challan or report is printed.

Table 2-61: Hardware Interfaces

Hardware	Applications	Interface requirement
Android phone or tablet	Admin Mobile, Teacher Mobile, Student Mobile	A touch display, enough local storage for the app, its Room cache, and any generated PDF, and network access. A camera or photo gallery is used only when a profile photo is set; no biometric reader or specialist

Hardware	Applications	Interface requirement
		sensor is required. Runs on Android 7.0 or later.
Desktop computer (Windows, on the JVM)	Admin Desktop, Teacher Desktop, Student Desktop	Keyboard and pointing-device input, a display suitable for responsive application windows, local storage, and network access. A Java runtime is required, since the desktop apps run on the JVM.
Printer	The three desktop apps, where Print is offered	Receives a user-approved print job, a fee challan or report, through the operating-system print dialog. The app stays usable when no printer is installed.
Local or removable storage	Apps that open, upload, save, or download files	A user-selected file or destination; the app never assumes access to a fixed personal folder.

2.5.4 Communications Interfaces

All communication is between the apps and the Supabase backend; no other external service sits in the loop.

Table 2-62: Communications Interfaces

Communication	Requirement
App and backend	All six apps exchange sign-in, college records, and files with the Supabase backend over an encrypted (HTTPS) connection, PostgREST for data, GoTrue for authentication, and Storage for files, carried over Ktor. An interrupted exchange leaves a retryable state, never a half-completed but confirmed action.
Notifications	Notifications are delivered in-app only, there is no push service (no FCM). Each one appears solely in the notification list of the users in its target role or academic audience.
Email	Only account-verification and password-

Communication	Requirement
	recovery messages are sent, through the backend's configured email (GoTrue); the apps send no SMS. Errors are reported without revealing whether another account exists, beyond what the current request needs.
File transfer	Uploads (such as an exam-paper PDF) and downloads preserve the file's content and clearly report progress, cancellation, an unsupported type, or failure.
Service keep-alive	A scheduled GitHub Actions job pings a lightweight endpoint every few days so the free-tier Supabase project is not paused for inactivity.

2.6 Summary

This chapter turned the project overview into a set of requirements the finished system can be checked against. Six user classes cover administrators, teachers, and students on both mobile and desktop; twenty fully dressed use cases and their diagrams show how each carries out their main work; and thirty-three functional requirements define the services and the failure behaviour expected of the system, two of them (the Documents feature) now withdrawn as the system no longer includes it. Reliability, usability, performance, security, and the external-interface requirements set measurable conditions across all six applications. Together these form the reference point for the design in Chapter 3 and for the testing and evaluation later in the report.

3. System Design

The College Management System is a family of six applications rather than one big interface: an admin, a teacher, and a student app, each on both a phone and a desktop, all sharing the same college records and business rules so that an action taken on one device shows up wherever that authorised information appears. Behind them sit managed services for accounts, the database, file storage, and email.

The rest of the chapter shows how those parts fit together, the design constraints and risks, the main information objects and how they relate, the architecture that ties the six apps to one backend, the data organisation, the user interfaces, the key interactions and workflow states, and the design decisions that keep mobile and desktop behaving the same.

3.1 Design Considerations

The design was shaped by the college environment: people move between office computers and personal phones, the internet is not always steady, and access to student information has to

follow each person's role. The main considerations, and the response built into the design, are below.

Table 3-1: Design Considerations, Constraints, and Risks

Category	Consideration	Design response
Assumption	Each user has one authorised account and uses the app meant for their role.	Sign-in verifies the account, role, and status before opening a workspace.
Assumption	Sessions, subjects, teachers, and rosters are maintained by authorised staff.	Dependent features reuse those approved records instead of asking users to re-enter the same information.
Dependency	All six apps rely on one shared backend (Supabase).	Shared validation and row-level access rules keep records consistent whatever device is used.
Dependency	File uploads, password recovery, and account verification rely on managed storage and email.	A failed external operation is reported clearly and never leaves a false success.
Constraint	Mobile screens are small; desktop users expect wider working areas and native file/print actions.	The information and actions stay the same; only the presentation is arranged for each device.
Limitation	Live records need a connection to confirm and share; the apps are offline-first.	Each app shows its last-synced cache when offline, while new or changed records wait for a confirmed connection.
Risk	A wrong role or permission rule could expose confidential academic information.	Access is enforced centrally in the database with Row-Level Security, not merely hidden in the interface.
Risk	Two users update related academic information at nearly the same time.	An important update is validated against the latest record and rejected when the context is no longer valid.
Risk	Deleting a parent record (a department or session) could remove a great deal of dependent data.	Deletes cascade by design and are gated behind a confirmation dialog that names what will be removed.

Category	Consideration	Design response
Risk	A file upload or download is interrupted.	Progress, cancellation, and failure are reported, and the college record is left unchanged when the action is incomplete.
Risk	A large first-time load could delay the administrative home screen.	Essential admin data is prepared at startup and reused by later screens instead of manual refreshes.
Risk	The free-tier backend pauses after a spell of inactivity.	A scheduled keep-alive ping keeps the project awake between uses.

3.2 Design Models

The design is captured in three complementary models, each taken up in a subsection below. The domain and class relationship model (§3.2.1) lays out the main information objects and how they connect. The interaction models (§3.2.2) trace the order in which a user, an app, and the shared backend cooperate to get something done. The state-transition models (§3.2.3) follow how approval and publication records move from one state to the next. Between them they cover structure, behaviour, and change without tying any of it to a particular programming language.

3.2.1 Domain and Class Relationship Model

The domain objects fall into people, academic structure, teaching activity, published information, and financial notices. A department contains academic sessions; a session holds its subjects, its enrolled students, its timetable, and its fee structure; teachers are attached through timetable periods (each in a building and room); and attendance, marks, and results tie a student to that teaching context. A user account provides authorised, role-specific access to whichever of these objects the role may see. Figure 3-1 shows the objects and their relationships, and Table 3-2 groups them by responsibility.

Table 3-2: Core Object Responsibilities

Object group	Key information	Main operations and inputs
Identity	Account role, status, contact details, and the approved link to a college person.	Verify an account at sign-in; open a workspace by role and status; link a student by account and roster identity; sign out.
Academic structure	Departments, sessions, semesters, subjects, rosters, buildings and rooms, and timetable periods.	Create a session from department, intake, and shift; assign a subject to a session and semester; schedule a period

Object group	Key information	Main operations and inputs
		from class, teacher, time, and room.
Teaching records	Attendance, assessment marks, mark-correction requests, and semester results.	Record attendance from class, date, roster, and statuses; enter marks from subject, assessment, and scores; review a correction from the request and decision.
College information	Calendar events, datesheets, examination papers, and notifications.	Prepare information with content and audience; publish after validation; open, save, or print using the selected record and device action.
Financial	Session fee structures, fee heads, per-student exceptions, and fines.	Build a challan view from session and student; issue a fine with student, amount, category, and reason; show financial notices without taking payment.
Reporting	Approved attendance, marks, results, session totals, and progress summaries.	Build a summary from role, academic scope, and filters; return only what the role is permitted to see.

> **Figure update required:** if Figure 3-1 (an image) shows a **Documents** class/entity, remove it, the Documents feature was dropped from the system. The published information now covers calendar events, datesheets, examination papers, and notifications only.

3.2.2 Interaction Models

The interaction (sequence) models are presented in Section 3.6. They cover signing in and role routing, an administrator publishing college information, a teacher marking attendance and entering marks, and a student registering and linking their account. Each one names the participant, the app in use, and the shared service involved, so the same logic reads the same way on mobile and desktop.

> **Figure update required:** the earlier set included Figure 3-16 "Document publication and opening." The Documents feature was removed, so that interaction no longer exists, drop Figure 3-16. Publishing college information (datesheets, notices, calendar events) is already covered by the administrative-publication interaction (Figure 3-13), so nothing is lost by removing it.

3.2.3 State-Transition Models

State models appear in Section 3.6 for the three workflows where the current state decides what may happen next and what other users can see: student account linking, mark-correction review, and publishing college information. A link request moves from pending to approved or rejected, and a later relink or delink downgrades a previous account's stale approval so a record keeps exactly one live link. A submitted mark is locked; a correction request against it stays pending until an administrator approves it, applying the new value, or rejects it, leaving the original in place. Publishable information is unpublished until it is released to its audience: a datesheet through an explicit published state, and calendar events and notifications by the audience they are addressed to.

> **Figure update required:** if Figure 3-19 (publishable-information states) includes a **documents** publication path, remove it, the Documents feature was dropped. The model should cover datesheets, calendar events, and notifications only.

3.3 Architectural Design

The system follows a layered client-server design. The six apps present role-specific workspaces on top of shared services that apply the same validation, access rules, and record operations, so the college's rules are written once rather than in six places. The managed backend holds the accounts, the college data, and uploaded files such as exam papers and photos. Each app also keeps a role-scoped local copy of the records it is allowed to see: it builds that copy on first use, and later refreshes pull only what changed since the last successful sync and apply it in one step, enough for a useful view on a weak connection, without becoming a second source of truth, since the online database stays authoritative.

The sections that follow set out the system context (§3.3.1, the six apps around the shared backend), the component structure (§3.3.2, the core and platform modules that feed the six apps, Figure 3-3), and the architecture style with the module mapping (§3.3.3, a layered MVVM/Repository design wired with Hilt, Figure 3-4).

3.3.1 System Context

At the edges of the system are its three kinds of user; at the centre is one shared backend. Administrators and teachers create and update information within their authority, while students mainly view their own and published information, with a few actions of their own, registration, account linking, and profile upkeep. Whether a person is on a phone or a desktop, their app exchanges data with the same backend, so moving between devices never creates a separate record. That backend is the Supabase project: PostgREST serves the college records, GoTrue handles sign-in, Storage holds uploaded files, and a small set of service-role functions carry out admin-only operations, all reached over HTTPS, with Row-Level Security deciding what each signed-in user may read or write.

> **Figure check:** Figure 3-2 should show the six apps around the **Supabase** backend (PostgREST / GoTrue / Storage / edge functions). If the image still labels the central service as Firebase or any earlier backend, update it to Supabase.

3.3.2 Component Structure

The code is organised into layers of components rather than six standalone programs. At the base sits the portable core module, the domain model, the data-transfer objects, the repositories, and the authentication logic every app shares. Two platform kits build on it: mobile-shared, the Android UI kit with the Room cache and the Hilt wiring the three phone apps use, and desktop-shared, the Compose-for-Desktop UI kit the three desktop apps use. The six app modules themselves are thin, each supplies its role's screens and assembles them from those shared parts. So a workspace in any app calls into a shared repository, which applies the identity, academic, publishing, and reporting rules before reaching the Supabase backend; the same rule runs the same way whichever app invoked it.

> **Figure note:** Figure 3-3 should show core under mobile-shared and desktop-shared, feeding the six app modules, with the shared repositories talking to Supabase. If it uses generic labels ("application shells", "managed online services"), align them to these real module names.

3.3.3 Architecture Style and Module Mapping

The apps follow a layered MVVM / Repository pattern, so each concern can be reasoned about and changed on its own: a Compose screen presents information and takes input, a view model holds the screen's state and decides what the user is doing, a repository applies the shared rules and reads or writes data, and the Supabase backend stores the authoritative records. On Android, Hilt wires these pieces together. The client-server line runs between the apps, each with its own local cache, and that backend.

Table 3-3: Architecture Layer Mapping

Layer	Responsibility	Where it lives
Presentation (View)	Renders role-appropriate Compose screens, takes input, shows progress and feedback, and adapts to a phone or a desktop.	mobile-shared and desktop-shared UI kits; per-app screens in the six app modules
View model	Holds screen state and drives the steps for the selected task, without owning the business rules.	Each app module (mobile-admin/teacher/student, desktop-admin/teacher/student)
Repository + domain	Applies identity, validation, permission, academic, publishing, and reporting rules, and mediates between the local cache and the backend.	core (shared by all six apps)
Local cache	Keeps a role-scoped offline copy of the records the user may see and serves it when the	Room, via mobile-shared

Layer	Responsibility	Where it lives
	network is down.	
Backend (data and services)	Stores the authoritative records and returns controlled results, Postgres through PostgREST, GoTrue for auth, Storage for files, with Row-Level Security enforcing access.	The Supabase project

3.4 Data Design

Information is organised around stable college records rather than around individual screens. Departments and academic sessions form the structure; people records identify administrators, teachers, and students; teaching records hold timetables, attendance, marks, and results; publication records carry datesheets, calendar events, and notifications; and financial records cover fees and fines. All six apps read and update the same authorised record, so a mobile and a desktop version never drift into separate sources of truth.

The relationships are explicit. A student belongs to an academic session, a subject is offered in a particular semester, and every attendance or assessment entry carries enough context to pin down the exact class. Published information records the audience it is meant for, and an approval record keeps both the requested change and the decision made on it.

Records also carry creation and update timestamps, which is how a refresh finds the newest authorised version of a row and pulls only what changed. Deletions are tracked long enough for other devices to drop their local copy, and ordinary lists never show removed rows. In the admin apps, an information control on each saved record can show its creation and update details, without exposing deletion details to routine users.

> **Figure note:** Figure 3-5 (Major data relationships) should not include a documents table, the Documents feature was removed. If the image shows one, remove it.

3.4.1 Data Dictionary

The dictionary lists the major information entities and calculated summaries in alphabetical order, each with its type and purpose. Field-level detail is left out on purpose, this chapter explains the design, not the column layout, and the calculated summaries are derived from the base records rather than stored as separate tables.

Table 3-4: Alphabetical Data Dictionary

Terminology	Description, type, and role
Academic Session	Record. One department intake and shift; holds the current-semester pointer and the status, and anchors the students, subjects, fees, and

Terminology	Description, type, and role
	timetable beneath it.
Administrator Account	Identity record. An authorised administrator, profile details, account status, and responsibility for approved actions.
Application Log	Operational record. Diagnostic entries the apps write for troubleshooting; not part of the academic data.
Assessment Mark	Academic record. A student's score for one subject and exam type, with the maximum (midterm 25, sessional 15) and the academic context it was recorded in.
At-Risk Student Summary	Calculated summary. Flags students whose attendance or standing falls below the college's review threshold, so authorised staff can follow up.
Attendance Record	Academic record. One student's present, absent, or leave status for a dated class meeting, with an optional late flag and note.
Attendance Summary	Calculated summary. Rolls class attendance up into present/absent/leave, total-marked, and percentage values.
Building	Master record. A named building on campus; its rooms are offered as a pick-list when a class period or exam is scheduled.
Calendar Event	Publication record. A holiday, event, exam period, or deadline, with dates and the audience allowed to see it.
Datesheet	Publication record. An examination schedule for one academic session and semester; a published flag controls whether that session's students can see it yet.
Datesheet Slot	Child record. One paper on a datesheet, subject, date, time, and a real building and room, with an optional invigilator.
Department	Master record. An academic department; the top-level grouping for sessions, teachers, and reports.

Terminology	Description, type, and role
Examination Paper Submission	Academic file record. Links a teacher's submitted mid-term paper (a PDF in Storage) to a published datesheet slot; administrators can view or download it, with no separate review step.
Examination Statistics	Calculated summary. Entered-mark counts, average and range, and pass rate for an assessment, within the viewer's authority.
Fee Exception	Financial record. An authorised adjustment to an individual student's standard session fee.
Fee Heading	Financial child record. One line of a session's fee structure, a label and amount.
Fee Structure	Financial record. The charges, due date, payment note, and annual-or-semester cadence for one session.
Fine	Financial notice. An informational fine on a student, category, amount, reason, date, and issuer. It records no online payment and no paid/unpaid status.
Mark Correction Request	Approval record. The recorded score, the requested score, the teacher's reason, the review state, and the reviewer's decision and time.
Notification	Communication record. An in-app title and message with its audience (a role, optionally narrowed to a department or session); read state is tracked per viewer. There is no push service.
Period-Session Assignment	Relationship record. Links a timetable period to one or more sessions when a class is shared (a merged lecture).
Room	Master record. A room within a building, chosen when a timetable period or datesheet slot needs a location.
Semester Result	Academic outcome. A student's semester GPA, CGPA, result status, class position, and any supply (repeat) subjects.
Semester Term	Academic date record. The opening and closing dates of a semester within a session.

Terminology	Description, type, and role
Session Overview	Calculated summary. Department, shift, semester, enrolment, attendance, and academic standing gathered into one session-level view.
Student	Person and enrolment record. Roll number, name, contact and guardian details, enrolment status, current standing, and the approved account link.
Student Account Link Request	Approval record. A registered account's claim to a student record, the claimed identity, the review state, and the final decision.
Student Progression Summary	Calculated summary. A student's semester-by-semester outcomes and changes in performance.
Subject	Academic record. A course code, name, credit value, theory-or-lab type, and the semester it is taught in, within a session's curriculum.
Teacher	Person and employment record. Identity, department, account status, contact details, and the delegated permissions an administrator has granted.
Timetable Period	Scheduling record. One slot on a session's weekly grid, day, time, period type, subject, teacher, and room. A database trigger prevents the same teacher or room being double-booked.
User Account	Identity record. The account's email, role, and status, and the approved person record used to open the correct workspace.

3.5 User Interface Design

The interface is organised around the work each role performs. Administrators see college-wide totals, academic structure, people, approvals, records, and reports; teachers see their assigned classes, attendance, assessment, schedules, students, and the college information they are permitted; students see only their own attendance, timetable, marks, results, fees, notices, and profile. On mobile the layout is compact, bottom navigation and vertically arranged cards, while the desktop uses the extra width for wider cards, tables, and file or print actions.

The wording and meaning of an action never change between devices: a teacher opening Mark Attendance goes through the same class, subject, roster, validation, and confirmation steps on a phone and on a desktop, and only the arrangement differs. That consistency is possible because

all six apps are built from one shared component library, the "Modernist" design system on Material 3, each app carrying its own signature accent. The subsections that follow show representative screens (§3.5.1), the objects and actions on two key screens (§3.5.2), and the feedback and interaction rules every screen shares (§3.5.3).

3.5.1 Screen Images

The mobile images are captured from working builds; the desktop images are representative views generated from the implemented desktop navigation and screen structure. Between them they show each of the six apps from its own role's perspective, all in the shared "Modernist" design.

3.5.1.1 Admin Mobile Application, the dashboard gives the administrator an immediate college summary and short routes into academic, people, record, and approval work. (*Figure 3-6: Admin Mobile dashboard*)

3.5.1.2 Admin Desktop Application, the same administrative areas, using a permanent navigation rail and wider information cards for office work. (*Figure 3-7: Admin Desktop representative interface*)

3.5.1.3 Teacher Mobile Application, the home screen leads with today's classes and the actions normally needed during a lesson, attendance and assessment. (*Figure 3-8: Teacher Mobile home screen*)

3.5.1.4 Teacher Desktop Application, the same assigned work with room for larger rosters, mark-entry tables, and schedules, plus desktop file and print actions. (*Figure 3-9: Teacher Desktop representative interface*)

3.5.1.5 Student Mobile Application, the home screen summarises the current semester and gives direct access to the student's own academic information, with no administrative controls. (*Figure 3-10: Student Mobile home screen*)

3.5.1.6 Student Desktop Application, the same personal information in a wider reading layout, with open, save, and print actions. (*Figure 3-11: Student Desktop representative interface*)

> **Figure check:** confirm each screenshot is from the current "Modernist" build. None should show a Documents area, the older visual design, or a Firebase sign-in screen, re-capture any that do.

3.5.2 Screen Objects and Actions

The two screens below show how visible objects, user actions, and feedback work together. The same objects appear in the matching mobile and desktop app; only their position changes with the space available.

3.5.2.1 Use Case: Mark Class Attendance

Table 3-5: Screen Objects and Actions for Mark Class Attendance

Screen object	User action	Displayed feedback
Class and subject selectors	Choose an assigned class	Only the teacher's own

Screen object	User action	Displayed feedback
	meeting and subject.	assignments are offered; missing context is explained before the roster opens.
Search field	Filter the roster by name or roll number.	The list narrows as the teacher types.
Student rows (Present / Absent / Leave segmented control)	Mark each student present, absent, or on leave, and flag a present student as late where needed.	The chosen state shows at once; late is an add-on flag, not a separate status.
Status summary strip	, (read-only)	Running present, absent, and leave counts update as marking proceeds.
Submit Attendance (sticky action)	Submit the completed register.	An incomplete or duplicate register is flagged; a successful save shows confirmation and the recorded date.
Retry (offline)	Retry after an interrupted connection.	Unsaved selections stay on screen, so the class does not have to be marked again.

3.5.2.2 Use Case: Review a Mark Correction

Table 3-6: Screen Objects and Actions for Review a Mark Correction

Screen object	User action	Displayed feedback
Pending request row	Open a request and review the student, subject, recorded mark, requested mark, and the teacher's reason.	The original and requested values stay visible side by side, so a decision is never made blind.
Approve action	Accept a valid correction (through a confirmation step).	The mark is updated, the request becomes approved, and it leaves the pending list.
Reject action	Reject the request, recording a reason where the screen asks for one.	The mark is left unchanged, the request becomes rejected, and the decision is kept.
Empty or failure state	Review the queue, or retry loading it.	The screen tells "no pending requests" apart from a connection or permission problem.

3.5.3 Common Feedback and Interaction Rules

Because every screen is assembled from the same component library, these feedback rules hold the same way across all six apps.

Table 3-7: Common Interface Feedback

Situation	Feedback provided across the six applications
Loading	Skeleton placeholders for lists and tables (a spinner for a first detail load) instead of blank space; a pull-to-refresh shows a slim progress bar over the existing content.
Empty	An empty-state panel explains that there are no records and, for authorised users, offers the action to create the first one.
Filtered-empty	When a search or filter matches nothing, the screen says so and offers to clear the filters, kept distinct from having no records at all.
Validation	The affected field and a plain-language correction are shown before submission, not after the whole form is rejected.
Success	A snackbar confirms the action and the visible information updates in place.
Connection failure	An offline banner explains that the sync did not complete and offers a retry; the last-synced cached data stays on screen, and no service address is shown.
Permission restriction	An action the role does not hold is hidden or disabled, and any direct attempt returns a clear access message.
Destructive action	A confirmation names the record and spells out the effect, including a cascade summary, such as how many dependent records go with it, before a delete, rejection, disable, or unlink.
Back and refresh	The top bar gives predictable back navigation and a refresh where live information may have changed.

3.6 Behavioural Model

Behavioural models show how the parts identified in the architecture cooperate over time. The interaction (sequence) diagrams trace the main cross-role journeys, signing in and role routing (§3.6.1), an administrator publishing college information (§3.6.2), a teacher recording attendance and assessment (§3.6.3), and a student registering and linking an account (§3.6.4), using application-neutral names, so each flow reads the same on the mobile and desktop version of a role. The state diagrams then follow how records change when a decision is made: a student link request (§3.6.6), a mark-correction request (§3.6.7), and publishable information (§3.6.8).

> **Note:** a former §3.6.5 covered a document publication-and-opening flow. The Documents feature was removed, so that subsection is withdrawn; publishing college information (datesheets, notices, calendar events) is covered by §3.6.2.

3.6.1 Sign-In and Role Routing

Sign-in follows the same short sequence in all six apps. The login screen hands the email and password to its view model, which calls the shared auth repository; that repository signs the user in through GoTrue and reads the account's role and status from its profile. Only when the account is active and its role matches the app does the matching workspace open, otherwise the screen shows a clear sign-in or access message, without exposing any service address. When the app is reopened, a still-valid session is restored and the same routing runs without asking for credentials again.

> **Figure check:** if Figure 3-12 labels the backend as "Firebase" or "Firebase Auth," update it to GoTrue (Supabase Auth).

3.6.2 Publish College Information

An administrator, or a teacher with the matching permission, composes an item in the admin or teacher app: a datesheet, a calendar event, a notification, or a fee or fine record. The view model checks the required fields and the chosen audience before anything is saved, and the repository then writes it to Supabase. Once it is published, only the roles or academic groups it is addressed to can see it in their own apps. The same flow serves every kind of published item, so they are all validated and stored the same way.

> **Figure check:** if Figure 3-13 includes a document publication path, remove it, the Documents feature was removed. The flow covers datesheets, calendar events, notifications, and fee/fine records.

3.6.3 Record Attendance and Assessment

A teacher opens an assigned class and subject in the mobile or desktop app, and its view model loads that class's roster from the shared records. For attendance, each student is marked present, absent, or on leave, with an optional late flag on a present student, and the completed register is submitted through the repository; only the assigned teacher can write attendance, while an administrator can read it for reports but never change it. For marks, the teacher enters a first-time score (midterm out of 25, or sessional out of 15) and submits it, after which it is locked. A later

change is never a silent overwrite: the teacher raises a correction request, and an administrator must approve it before the recorded mark moves.

> **Figure check:** Figure 3-14 should show attendance written only by the assigned teacher, and a submitted mark locking (with correction routed through admin approval).

3.6.4 Register and Link a Student Account

A student registers in the mobile or desktop app through GoTrue, then submits a link request to claim their college record: a roll number chosen from the list of unclaimed rolls for their session, together with identity details (CNIC/B-form, name, and date of birth) for a reviewer to check. Until that request is approved the account stays gated, attendance, marks, results, fees, and timetable are all out of reach. An administrator, or a teacher granted the permission, reviews the request and approves it, linking the account to exactly one student record; if that record is later re-linked or delinked, the previous account's stale approval is downgraded so a record never carries two live links.

> **Figure check:** Figure 3-15 should show the account gated until approval and the admin/permitted-teacher approval step that performs the link.

3.6.5 Publish and Open a Document (Withdrawn)

Withdrawn. This interaction described publishing a document and a user opening it. The Documents feature was removed from the system (20260827190000_drop_documents_feature_table.sql; Room MIGRATION_30_31), so the flow no longer exists.

Recommendation: delete §3.6.5 and Figure 3-16. Publishing college information, datesheets, notices, and calendar events, is already covered by §3.6.2 and Figure 3-13, so nothing is lost. The §3.6.5 heading is retained here only to avoid renumbering the state-diagram subsections that follow (§3.6.6-§3.6.8).

3.6.6 Student Link Request States

A student account starts unlinked. Submitting a claim moves it to pending, and an authorised reviewer either approves or rejects it. Approval opens the student's personal workspace; rejection keeps academic information closed but lets the student send a corrected request. An unlink, or a re-link to a different account, removes access without deleting the college's student record, and it downgrades the previous account's now-stale approval so the record is never left with two live links. The app stays gated in every state except approved.

> **Figure check:** Figure 3-17 should show pending to approved / rejected, the resubmit path from rejected, and the unlink/relink transition that downgrades a prior approval.

3.6.7 Mark Correction Request States

A saved mark is locked. A teacher's correction request against it creates a pending review, and only one request can be pending for a mark at a time. An administrator, never the teacher who raised it, approves the request, which applies the proposed value, or rejects it, which leaves the

original mark unchanged. Either way the decision is final and kept, so the mark's history can be traced later.

> **Figure check:** Figure 3-18 should show locked to pending to approved (proposed value applied) / rejected (original kept), with at most one pending request per mark and the decision made by someone other than the requester.

3.6.8 Publication States

A datesheet begins as a draft while its papers are entered, and stays hidden until it is published; only then can its session's students see it. Calendar events and notifications have no separate draft step, they become visible to their target audience as soon as they are created. When any of these is no longer current, it can be archived or removed from active views without touching unrelated college records.

> **Figure check:** Figure 3-19 should cover datesheets (draft to published), calendar events, and notifications only. Remove any documents publication path, the Documents feature was removed.

3.7 Design Decisions

The main design choices follow from the college's roles, the sensitivity of academic records, and the need to support both classroom phone use and longer office work. Each is a standing choice with a reason and an accepted trade-off.

Table 3-8: Major Design Decisions

Design choice	Reason for the choice	Accepted trade-off
Separate Admin, Teacher, and Student apps, built from shared modules	Each role gets a smaller, clearer workspace, and sensitive admin actions stay out of student and teacher navigation; the apps reuse one shared component library and rule-set.	Six packages to build, test, and release, though only one library and one set of rules sit behind them.
A mobile and a desktop version of every role	Phones suit work during class and around campus; the desktop suits larger records, printing, and office tasks.	Each layout needs a device-specific arrangement even though the behaviour stays identical.
Access rules enforced in the database (Row-Level Security)	Confidential records stay protected even if a user bypasses a hidden screen or calls the backend directly, access is a property of the data, not just the UI.	Each new record type needs its access policies written and tested in the database.
One relational, normalised schema on Postgres with real	Departments, sessions, subjects, students, and teaching records	A report sometimes joins several related records before it

Design choice	Reason for the choice	Accepted trade-off
foreign keys	have explicit relationships and no duplicated facts; a mark cannot attach to a student who was never enrolled.	can show a summary.
One shared source of truth, with an offline-first local cache	A record changed by an authorised user is what every permitted app sees, with no manual transfer; a Room cache keeps recent data usable on a weak connection, and sync pulls only what changed.	Confirming a new or changed record still needs a connection, and a few low-frequency screens need to be online.
Layered MVVM / Repository architecture	Presentation, role workflow, shared rules, and storage can each be understood and changed on their own.	A small action passes through several layers before it is confirmed.
Lock-then-approve for marks, with a single controlled write-path for GPA	A submitted mark cannot be changed silently, and a semester GPA is set only through one reviewed path, so results stay trustworthy.	A correction waits for an administrator, so it takes longer than a direct edit.
Status-based approvals for linking, corrections, and publication	Pending work and final decisions stay visible and auditable.	Work waits on an authorised reviewer.
In-app notifications only	Announcements reach the right audience inside the apps, without depending on an external push service or extra device permissions.	A user sees a notification the next time they open the app, not as a device push.
Administrative startup preparation	Core reference data loads when the Admin app opens, cutting repeated manual refreshes.	The first startup does more background work.
Native desktop file and print actions	Desktop users open, save, and print through familiar operating-system dialogs.	Those native windows follow the OS look, not the app's own design.

3.8 Summary

This chapter turned the requirements into a full design for the six applications, Admin, Teacher, and Student, each on mobile and desktop. The domain, context, component, layered, and data

models set out the structure; the interface views and screen-action tables showed how each role does its work and gets feedback; and the interaction and state-transition diagrams traced the main behaviours, sign-in and role routing, administrative publication, teaching (attendance and marks), student registration and linking, and the approval and publication state machines. The design decisions balance role separation against a single shared rule-set, consistent mobile and desktop behaviour, access enforced in the database, clear academic relationships, offline-first working within connection limits, and familiar device services. These models are the reference for the implementation in Chapter 4.

4. Implementation

This chapter shows how the main features actually work, without reprinting source code. The key algorithms are written as language-neutral pseudocode, so their decisions, inputs, outputs, and cost can be judged on their own rather than through Android or desktop specifics (§4.1). It then records the external services the six apps depend on (§4.2) and the current state of the shared Git repository, including contributor activity (§4.3). The coding standards the project follows are collected in Appendix B.

4.1 Algorithm

The algorithms below cover the work shared across all six apps and the role-specific work that matters most to administrators, teachers, and students. Each is given as language-neutral pseudocode with a rough cost, where **N** is the students in a roster, **P** the timetable periods checked for a conflict, **S** the candidate students in a session, **M** the published records scanned, and **F** the size of a file.

Table 4-1: Algorithm Coverage Across the Six Applications

#	Algorithm	Primary applications	Main purpose
1	Start and route a signed-in user	All six	Restore a valid session and open only the workspace for the account's role.
2	Prepare administrative data	Admin Mobile, Admin Desktop	Load the core college reference data once for the signed-in administrator.
3	Validate a timetable period	Admin Mobile, Admin Desktop	Prevent teacher, room, and session clashes before a period is saved.
4	Record class attendance	Teacher Mobile, Teacher Desktop	Validate and save one complete dated register.
5	Record assessment	Teacher Mobile,	Validate first-time

#	Algorithm	Primary applications	Main purpose
	marks	Teacher Desktop	scores and lock them once submitted.
6	Review a mark correction	Teacher and Admin (mobile and desktop)	Route a correction through a visible request and an authorised decision.
7	Link a student account	Student and Admin (mobile and desktop)	Match a registered account to one approved student record.
8	Build a student academic overview	Student Mobile, Student Desktop	Combine the student's own attendance, schedule, marks, results, and fees.
9	Publish information for an audience	All six, according to role	Publish and retrieve notices, datesheets, and calendar events for the permitted audience.
10	Complete a desktop file action	Admin, Teacher, and Student Desktop	Open, save, export, print, or share an authorised file through the operating system.

4.1.1 Start and Route a Signed-In User

Every app runs the same startup check, so a user lands in the right place, or is stopped, without ever seeing something they shouldn't. It reuses a valid session where one exists, confirms the account's role and status before opening a workspace, and, for an administrator, kicks off the data preparation in Algorithm 2.

Algorithm 1: Start and route a signed-in user

- *Input:* a saved account session and the selected role app
 - *Output:* the login screen, an access message, or the authorised role workspace
 - *Cost:* $O(1)$, a few decisions and indexed account lookups
1. Show the splash screen while the account check is in progress.
 2. Restore the saved session if one exists.
 3. If there is no valid session, open the login screen and stop.

4. Read the cached role, then confirm the current role and account status from the account's profile.
5. If the account is inactive, show an access message, clear the session, and stop.
6. If the role does not match the selected app, show the correct-role message and stop.
7. If the role is Administrator and the core data is not yet prepared, run Algorithm 2.
8. Open the authorised home workspace and stop.

4.1.2 Prepare Administrative Data

When the Admin app opens, it loads the college's core reference data once in the background rather than fetching it screen by screen. The dashboard shows progress and whatever is already cached, so the administrator is never left on a blank page; later refreshes pull only the rows that changed.

Algorithm 2: Prepare administrative data

- *Input:* the administrator account and its saved preparation status
 - *Output:* a ready workspace built on current or previously prepared data
 - *Cost:* $O(D + T + S + R)$ over the departments, teachers, students, and supporting records loaded
1. Start once the administrator role is verified.
 2. If preparation is already complete for this account, open the workspace and stop.
 3. If usable prepared data already exists, mark it ready, open the workspace, and stop.
 4. Show a preparation message and block duplicate preparation attempts.
 5. Request departments, sessions, teachers, students, timetable, approvals, and supporting records.
 6. Validate each returned group before it replaces the previously available data.
 7. If the essential groups load, remember completion for this administrator.
 8. If any request fails, keep the safe existing data and offer a retry.
 9. Open the administrative workspace and stop.

4.1.3 Validate a Timetable Period

Adding a period to a session's timetable has to avoid two clashes: the same teacher, or the same room, booked over the same time on the same day. The check runs in the app and, decisively, as a **BEFORE trigger in the database**, so a conflict is caught even if two edits race. Only lecture periods are checked; breaks, zero periods, and a merged lecture (one row serving several sessions) are exempt. Rooms are picked from the managed buildings-and-rooms lists.

Algorithm 3: Validate a timetable period

- *Input:* session, day, start/end time, teacher, room, and effective dates

- *Output*: the saved period, or a clear conflict message
- *Cost*: $O(P)$ over the P relevant periods on that day

1. Start when a timetable period is submitted.
2. If the start time is not before the end time, reject the period and stop.
3. For each existing lecture on the same day whose effective dates overlap (a null date means unbounded):
 - a. If its teacher matches and the times overlap, reject with the teacher conflict and stop.
 - b. If its room matches and the times overlap, reject with the room conflict and stop.
4. Check that the session does not already hold the same slot.
5. Save the period, attach any approved shared sessions (a merged lecture), confirm, and stop.

Time overlap is half-open, two periods clash when one starts before the other ends and vice versa.

4.1.4 Record Class Attendance

Only the teacher assigned to a class can record its attendance, the database's row-level rules enforce that, and an administrator can read the result but never write it. The teacher marks each student present, absent, or on leave, optionally flagging a present student as late, and the whole register is submitted in one go; a register already recorded for that meeting is opened for review rather than silently duplicated.

Algorithm 4: Record class attendance

- *Input*: the assigned class, subject, date, roster, attendance states, late flags, and notes
 - *Output*: one confirmed attendance entry for every student in the class
 - *Cost*: $O(N)$ time and space over N students
1. Start after the teacher selects an assigned class and subject.
 2. Load the roster and any attendance already recorded for that date (open it for review if it exists).
 3. For each student, capture present, absent, or leave, plus an optional late flag and note.
 4. If any student's status is missing, point out who and stop without saving.
 5. Build one dated entry per student.
 6. Submit the complete register as a single operation.
 7. If it is confirmed, update the visible history and show success.
 8. Otherwise keep the teacher's selections and offer a retry.

4.1.5 Record Assessment Marks

A teacher enters marks for one subject and assessment type, midterm out of 25, or sessional out of 15 (its attendance, assignment, and quiz components of five each, summing to fifteen). A

score must be a whole number within that maximum; anything blank, non-numeric, negative, or over the limit is flagged before submission. Once submitted, a mark is locked, and any later change is not a direct edit, it goes through the correction request in Algorithm 6.

Algorithm 5: Record assessment marks

- *Input:* the assigned class, subject, assessment type, roster, scores, and absence selections
 - *Output:* confirmed first-time marks, or field-level correction messages
 - *Cost:* $O(N)$ time and space over N students
1. Start after the teacher selects an assignment and assessment type.
 2. Load the roster, any marks already recorded, and any pending correction requests.
 3. For each student without a locked mark:
 - a. accept "absent" or a whole-number score within the assessment maximum (25 for midterm, 15 for sessional);
 - b. show a correction message for a blank, non-numeric, negative, or too-large value.
 4. If any entered value is invalid, keep the form open and stop.
 5. Save only the valid first-time marks and lock them once confirmed.
 6. Route any later change to Algorithm 6 (mark correction) and stop.

4.1.6 Review a Mark Correction

Changing a locked mark is a two-person job: a teacher raises a correction, and an administrator, never the same teacher, decides it. That is what keeps a submitted score from being altered quietly.

Algorithm 6: Request and review a mark correction

- *Input:* the locked mark, the requested mark, a reason, the teacher's identity, and the administrator's decision
 - *Output:* an approved, updated mark, or a rejected request with the original retained
 - *Cost:* $O(1)$ per request
1. Start when a teacher selects a locked mark.
 2. Check the requested value is a whole number within the assessment maximum.
 3. If a request is already pending for this mark, show its status and stop, only one may be open at a time.
 4. Create a pending request holding the original value, the requested value, and the reason.
 5. Show it in the review list of an authorised administrator, who cannot be the teacher who raised it.
 6. If the administrator approves, apply the requested value to the mark and close the request as approved.

7. Otherwise leave the mark unchanged and close the request as rejected.
8. Record the reviewer and the decision time, a decided request cannot be decided again, refresh the affected view, and stop.

4.1.7 Link a Student Account

A registered student sees no academic data until their account is linked to a college record, the app stays gated until then. The student claims a roll number from the list of unclaimed ones in their session and adds identity details (CNIC/B-form, name, and date of birth); an administrator checks the claim against the roster and links exactly one account to one record, keeping a single live link per record.

Algorithm 7: Link a student account

- *Input:* the registered account, the selected department and session, a claimed roll number, and supporting identity (CNIC, name, date of birth)
 - *Output:* an approved personal workspace, or a rejected request with a reason
 - *Cost:* $O(S)$ over the S students in the selected session
1. Start once student registration is verified.
 2. If the account is already linked, open the personal workspace and stop.
 3. If a request is already pending, show its status and stop.
 4. Let the student pick a roll number from the session's unclaimed list and submit it with their identity details as a pending request.
 5. An administrator reviews the request against the session's roster, seeing only the claimed roll and matching identity, not other students' details.
 6. If exactly one authorised record matches, link the account to it and mark the request approved; if that record was linked to another account before, downgrade the previous account's stale approval so only one live link remains.
 7. Otherwise reject with a clear reason and keep the academic records closed, the student may resubmit.
 8. Refresh the student's access state and stop.

4.1.8 Build a Student Academic Overview

A student's home brings everything about their own record into one view. It is assembled on the device from the base records the student is allowed to see, the same row-level rules that protect the data also scope it, rather than from a prepared server summary. Absent or unpublished values are shown as such, never as a misleading zero.

Algorithm 8: Build a student academic overview

- *Input:* the approved student link and the current session
- *Output:* a home summary, attendance, schedule, marks, results, fees, and notices

- *Cost*: $O(C + A + M + R)$ over the courses, attendance entries, marks, and result records used
1. Start once the student account is linked.
 2. Load the student's current session, semester, subjects, and timetable.
 3. Load only that student's own attendance, marks, semester results, fees, fines, and the publications meant for them.
 4. Work out per-subject attendance percentages (against the 75% line) and find the next scheduled class.
 5. Take current standing, GPA, CGPA, and any supply subjects, from the latest completed result.
 6. For marks, show midterm out of 25, sessional out of 15, and the total, marking an unentered or absent value plainly rather than as zero.
 7. If one information group is unavailable, show the rest and a retry for the missing group.
 8. Present the same information in the mobile or desktop layout and stop.

4.1.9 Publish Information for an Audience

Most of what an administrator or teacher creates is meant for some people and not others: a datesheet for one session, a calendar event for a department, a notice for a single class, a fine against one student's fee record. The publisher never picks who can see an item by hand. They set what it is and who it is for, a role, and where it applies a department or a session, and the same row-level rules that guard the database decide, per reader, whether the item is returned at all. A datesheet is the one type that waits: it is saved as a draft and only reaches students once it is explicitly published. A calendar event, a notification, or a fee record is live to its audience the moment it is written.

Algorithm 9: Publish information for an audience

- *Input*: the authorised publisher, the item type, its fields, and the target audience (role, and optionally a department or session)
 - *Output*: an item stored and returned only to the readers it is meant for
 - *Cost*: $O(M)$ over the M records a given reader is entitled to list
1. Start when an authorised administrator or teacher composes an item.
 2. Check the required fields for that type, a datesheet needs its session and slots, a calendar event its date, a notification its title and body, a fine its amount and the fee it attaches to.
 3. Confirm the audience is one the publisher may write to: an administrator across their department, a teacher within a class they are assigned.
 4. Write the item with its audience recorded on the row, the role it targets, and the department or session that scopes it.
 5. For a datesheet, hold it as a draft; it stays out of student view until the publisher marks it published. Other types are visible to their audience on write.

6. When an app lists current items, the reader's own role and academic scope are applied by the row-level rules before anything is returned, no client-side filtering stands between a reader and a record they should not see.
7. Show an empty list as empty, and tell it apart from a request that failed.
8. Let the publisher amend or withdraw an item within the same authority that let them create it, and stop.

4.1.10 Complete a Desktop File Action

On the desktop apps a user can take a record they are already looking at and turn it into a file on their own machine, a CSV or PDF of an attendance report or roster, or a printable fee challan. The app builds the file in memory, hands the operating system a native dialog to pick where it goes or which printer prints it, writes or prints the result, and opens it in the default viewer. None of this changes anything in the college database; an export only reads what the signed-in role is already allowed to see, so a cancelled or failed action leaves the records exactly as they were.

Algorithm 10: Complete a desktop file action

- *Input*: the record on screen, the chosen action (save, print, or share), and the destination the OS returns
 - *Output*: a saved, printed, or opened file, or a clean cancellation, with college data untouched either way
 - *Cost*: $O(F)$ over the F bytes of the file; the source records are already loaded
1. Start when a desktop user chooses to export, print, or share what is on screen.
 2. Build the payload from the loaded records, a CSV or PDF report, or a three-copy challan PDF. The data was fetched under the role's own access rules, so nothing outside that scope can enter the file.
 3. Open the matching native dialog: a save-file dialog for an export, the OS print dialog for a print, the default handler for a share.
 4. If the user cancels the dialog, stop, no file is written and no college data changes.
 5. Write the file to the chosen location (or send it to the selected printer), then hand it to the OS so the default viewer opens it.
 6. If the OS action succeeds, show the confirmation.
 7. Otherwise report it in plain language, no permission for that folder, the printer is unavailable, the file could not be written, and stop.

4.2 External APIs/SDKs

The six apps lean on a small, deliberately shared set of libraries rather than a different stack per platform. Everything to do with the backend, sign-in, reading and writing records, files, and the privileged server operations, goes through one client: the Supabase Kotlin SDK, version 3.1.4, pulling in its PostgreSQL, Auth (GoTrue), Storage, Realtime, and Functions modules from a

single BOM. That SDK runs on Ktor 3.1.1 for the actual HTTP, and Kotlinx Serialization 1.8.0 turns records into JSON on the way out and back. There is no Firebase or FCM anywhere in the project; notifications are rows the apps read under their own access rules, not a push service.

Above that, the split is only where it has to be. The three phone apps draw their screens with Jetpack Compose (Material 3), the three Windows apps with Compose Multiplatform (Material 3), the same declarative model, two runtimes. Both sides cache their data with Room so a workspace reopens with its last good records offline; the desktop build sits on a slightly newer Room (2.8.4) with the JVM bundled SQLite driver because it targets plain JDBC rather than Android's SQLite. Dependency injection follows the same pattern, Hilt on Android, its Dagger core on the desktop, both 2.52. PDF generation is the one place the platform APIs genuinely differ: the desktop draws its fee challans and report exports with Apache PDFBox, while the phones use Android's own PdfDocument, so that capability lives once per platform rather than as shared code.

Service addresses and keys are supplied through protected build configuration, they are not in the report or the repository, and the messages shown to users name the action that failed without exposing a service address or request detail.

Table 4-2: Details of External APIs and SDKs

API / SDK	Version	Purpose in CMS
Supabase Kotlin SDK (postgrest-kt, auth-kt/GoTrue, storage-kt, realtime-kt, functions-kt)	3.1.4	The single backend client, across all six apps: verifies sign-in and restores sessions; selects/inserts/updates/deletes records under row-level security; uploads and downloads exam-paper and photo files; and calls the privileged edge functions (create account, change teacher status, promote or archive a session, revoke a student link).
Ktor Client	3.1.1	The HTTP engine the Supabase SDK runs on, OkHttp on Android, CIO on the desktop. All six apps.
Kotlinx Serialization	1.8.0	Encodes and decodes the JSON exchanged with PostgREST, mapping DTOs to and from records. All six apps.
Kotlinx Coroutines	1.9.0	Keeps network calls, database work, file writes, and startup off

API / SDK	Version	Purpose in CMS
		the UI thread; structured launch/await/cancel. All six apps.
Jetpack Compose (BOM) + Material 3	2024.12.01	The declarative UI toolkit for the three phone apps, screens, navigation, lists, forms, dialogs.
Compose Multiplatform + Material 3	1.7.3	The declarative UI toolkit for the three Windows desktop apps, windows, wide role workspaces, shared components.
Room + androidx.sqlite bundled driver (desktop)	2.6.1 mobile · 2.8.4 + 2.6.2 desktop	On-device cache of the records a role may see, so workspaces read offline and refresh incrementally. Mobile on Android SQLite; desktop on the JVM bundled SQLite driver.
DataStore Preferences	1.1.1	Stores small non-academic app state, last notification-seen time, first-run/sync flags. Android on all phones; the pure-JVM core on the desktop.
Coil	2.7.0	Loads and caches profile and content images in the Compose UI. Three mobile apps.
Hilt (mobile) / Dagger (desktop)	2.52	Assembles each screen and service from its platform's shared components.
Apache PDFBox	3.0.3	Generates three-copy fee challans and report exports on the desktop. Desktop apps. (Phones use Android's built-in PdfDocument for the same output.)

4.3 Code Repository

The whole project lives in one Gradle repository, tracked in Git and hosted on GitHub. settings.gradle.kts wires it into nine modules: the three mobile apps and three desktop apps, plus

the three they share, core (the portable domain, DTOs, direct-PostgREST repositories, and auth), mobile-shared (the Android UI kit with Room and Hilt), and desktop-shared (the Compose-Desktop UI kit and its repositories). The database itself is versioned alongside the code: the Supabase migrations and edge functions sit under supabase/, so a schema change and the app code that depends on it move together in the same commit. Build configuration, tests, the documentation sources, generated diagrams, and the final report are all in the same tree.

Keeping everything in one repository is the point, not an accident of convenience. A change to shared behaviour, a repository method, a DTO field, a migration, shows up immediately in every app that depends on it, rather than drifting out of sync across separate projects.

Git Repository Link: <https://github.com/hadiyasaleem/CMS>

The two sections that follow report the repository's numbers. §4.3.1 covers the commit, branch, and integration metrics; §4.3.2 breaks down what each contributor committed. Because the local history runs ahead of the last public push, local counts and GitHub-side counts are given separately rather than merged into one figure that would fit neither.

4.3.1 Metrics of the Git Repository

The numbers below are a snapshot of the local history as of the last commit (2026-09-10). Everything under "measured from the repository" was read straight from Git at that point, commit count, file counts, and line counts, so it describes the code that actually exists, not an estimate. The GitHub-side collaboration figures are reported separately, and as last recorded on the public repository: the local history runs ahead of the last public push, so folding the two into one total would misrepresent both.

Table 4-3: Git and GitHub Repository Metrics

Metric	Value
Commits (reachable from master)	327
History span	2026-01-14 to 2026-09-10
Contributors (people, after consolidating aliases)	3
Branches (local and on origin)	6, master, three per-member development branches, and two feature/refactor branches
Gradle modules	9 (six apps + core, mobile-shared, desktop-shared)
Tracked files	1,114
Kotlin source files	649
Kotlin source lines	about 70,900
SQL migrations under supabase/migrations/	25 (about 1,600 lines)
Supabase edge functions	6

Metric	Value
<i>GitHub-side, as last recorded on the public repository (not re-measured in this snapshot):</i>	
Pull requests	20 total, 20 merged, 0 open
Issues	0
Pull-request reviews	1

Two things are worth reading off this table. The 327 commits come from three people over roughly eight months, and the per-person split is in §4.3.2, the contributor count here is people, not raw author strings, because the same person committed under several name/email spellings that are consolidated back together. And the shape of the codebase is mostly Kotlin: about 71,000 lines across 649 files, sitting on 25 SQL migrations and six edge functions that version the database and its privileged operations alongside the app code.

4.3.2 Contributor Statistics

Three people wrote the project, and each of them committed under more than one name/email spelling over the course of it, different machines, a mix of full name and handle. A .mailmap and a manual pass fold those aliases back to the right person, so one contributor's work shows up as one row rather than four. The counts below are the commits reachable from master (the same 327 reported in §4.3.1), split by author.

Table 4-4: Consolidated Contributor Statistics

Contributor	Reg. No.	Commits	Main areas
Sharfa Kiran	085646	117	Shared UI kits (mobile-shared, desktop-shared), the teacher apps, and core domain, the widest spread across the codebase
Hadia / Hadiya Saleem	085668	147	Student and teacher mobile apps and the shared modules they build on
Syeda Laraib Qamar Kazmi	085713	63	Admin apps, plus shared modules and core
Total		327	

One caveat on reading this table: a commit count is who committed a change, not a scoring of effort. Commits vary from a one-line fix to a whole feature, and generated build output was checked in at points early on, which would inflate any line-added/removed figure, so the split is reported as commits and focus areas rather than line churn. There are no automated or co-authored commits in the history: every commit is by one of the three team members, none by a bot or a Co-Authored-By trailer.

4.4 Summary

This chapter described the implementation as ten algorithms rather than source listings, shared startup, administrative setup and scheduling, teacher attendance and assessment, the two-person mark correction, student linking and the personal academic overview, audience-scoped publication, and desktop file actions. Each is language-neutral pseudocode that maps onto the same code across the six apps. The external-services table then named the libraries every app relies on and what each is used for, from the single Supabase client down to the platform PDF facilities. The repository section closed the chapter with the numbers measured from the live history, commits, files, and modules, and the consolidated split across the three contributors, keeping the local totals separate from the public-push figures. Together these give a traceable link from the Chapter 3 design to the testing in Chapter 5.

5. Testing and Evaluation

5. Introduction

Because all six apps share one core of academic rules, most of the testing effort points at that shared logic: a mistake in how attendance percentages, marks limits, or sync deltas are computed would show up in every app at once, so that is where a check earns its keep. The delta-sync logic, the part that decides what has changed since the last pull, carries the substantive automated coverage; the rest of the discipline is scenario-based. A per-screen registry lists, for each role and route, the states a screen has to handle correctly (loading, empty, error, validation, permission, offline), and that list is worked through against both the mobile and desktop builds to confirm the two platforms behave the same way from the same data.

The sections that follow take this in order: unit testing of the shared rules (§5.1), functional testing of the role workflows (§5.2), integration testing across the app-to-Supabase boundary (§5.3), and performance (§5.4), which is reported honestly as a local baseline on development hardware rather than a load test at scale. §5.5 draws the results together.

5.1 Unit Testing (UT)

The automated unit tests target the one piece of logic where a silent error would be both easy to make and hard to notice: incremental sync. Every app pulls only the rows that changed since its last successful sync, using a stored checkpoint (an `updated_at` watermark per owner, table, and scope), and then merges that delta into the local cache. Get the paging or the checkpoint advance slightly wrong and the app quietly misses records or re-applies stale ones, so this is the code that carries real unit coverage, in `IncrementalSyncTest` under `core`. The tests use a fake checkpoint store and a scripted fetch, so they run with no network and no database.

The domain validation rules, a midterm score in 0-25, a sessional in 0-15, an attendance register that must be complete before it locks, a mark edit that only an administrator other than the author may approve, are enforced in the shared core code and checked through the functional and parity testing in §5.2 rather than through separate unit tests. This is the honest state of coverage: unit testing is deliberately narrow, aimed at the sync engine, not a wide harness over every rule.

Table 5-3: UT-1: Paged delta fetch advances the checkpoint

Field	Detail
ID	UT-1
Scenario	Fetch a change delta across several pages starting from the stored checkpoint
Input	A checkpoint at 10:00:00Z; three rows returned over two pages (page size 2)
Expected	All three rows are returned in order; each page is requested from the inclusive checkpoint; the checkpoint advances once to the newest row's timestamp (10:02:00Z)
Result	Pass

Table 5-4: UT-2: A failed sync leaves the checkpoint untouched

Field	Detail
ID	UT-2
Scenario	A page fetch fails, or applying the delta to the local cache fails, part-way through a sync
Input	A stored checkpoint; a fetch (or a cache write) that throws on the second step
Expected	The error propagates; the checkpoint object is unchanged and never upserted, so the next sync safely re-pulls from the same point
Result	Pass

Table 5-5: UT-3: Delta merge by stable key

Field	Detail
ID	UT-3
Scenario	Merge a fetched delta into the existing cached rows, keyed on stable id

Field	Detail
Input	Two cached rows; a delta that tombstones one, updates another, and adds a new one
Expected	The tombstoned row is dropped, the updated row keeps its newer timestamp, and the new row is added, the result reflects the delta, not a blind append
Result	Pass

5.2 Functional Testing (FT)

Functional testing works through the role workflows end to end, the paths a real admin, teacher, or student actually takes, and confirms the visible result matches the rule behind it. These are scenario checks run against controlled sample data (a prepared department, session, roster, and marks) rather than automated tests, so each one records its steps and its pass condition and can be repeated from the same starting data. The four below span the three roles and the requirements they trace to; the same shared logic sits under both the mobile and desktop builds, so a case that passes on one platform is confirmed on the other through the parity checklist.

Table 5-6: FT-1: Sign in and role routing (FR-1)

Field	Detail
ID	FT-1
Scenario	A user signs in and lands in the workspace for their role
Steps	1. Enter valid credentials. 2. Submit sign-in. 3. Observe the landing workspace.
Expected	The session is verified and the user is routed to their own role's home, admin, teacher, or student, with only the screens and data that role may see; a wrong credential is refused with a plain message and no routing.
Result	Pass

Table 5-7: FT-2: Record attendance (FR-10)

Field	Detail
ID	FT-2
Scenario	A teacher marks a class register for a scheduled

Field	Detail
	period
Steps	1. Open an assigned class. 2. Mark each student PRESENT, ABSENT, or LEAVE, setting the late flag where needed. 3. Submit the register.
Expected	Every student must carry a status before the register submits; a late student is saved as present with the is_late flag, not as a separate status; the saved register reappears correctly in attendance history.
Result	Pass

Table 5-8: FT-3: Enter marks, then correct a locked mark (FR-12 / FR-13 / FR-14)

Field	Detail
ID	FT-3
Scenario	A teacher submits marks, then raises a correction that an administrator decides
Steps	1. Enter midterm (0-25) or sessional (0-15) scores for the roster and submit. 2. Attempt to change a submitted score. 3. Raise a correction request with a reason. 4. As a different administrator, approve it.
Expected	Out-of-range or non-numeric scores are refused at entry; a submitted mark is locked and cannot be edited directly; the correction is applied only after an administrator, never the teacher who raised it, approves, and the original is retained until then.
Result	Pass

Table 5-9: FT-4: Link a student account (FR-27 / FR-28)

Field	Detail
ID	FT-4
Scenario	A student claims a roll number and an administrator approves the link
Steps	1. As a registered student, claim an unclaimed

Field	Detail
	roll in the session and submit identity details. 2. As an administrator, review the pending request. 3. Approve the matching request.
Expected	The student sees no academic data until approval; the admin sees only the claimed roll and matching identity; approval links exactly one account to one record and, if that record was linked before, downgrades the previous stale link so only one live link remains.
Result	Pass

5.3 Integration Testing (IT)

Integration testing looks at the seams, the points where an app hands off to Supabase and where two apps meet through the shared database. Three of those seams matter most. Access control is not enforced in the app; it is enforced by Row-Level Security in Postgres, so the test that counts is whether the database itself refuses an out-of-role read or write even when a client asks for it. The second is the offline-first path: a record written in one app has to reach another app's cache correctly through the delta sync, without a manual refresh. The third is platform parity, the mobile and desktop builds share the same logic, and the per-screen scenario registry is the checklist that keeps their behaviour aligned.

To be clear about what was exercised where: RLS and the app-to-Supabase calls were checked against the live project with role-specific accounts; the delta-sync reconciliation has automated coverage over its logic (§5.1) and was confirmed end to end on the live data; and parity was verified by working the scenario registry against both builds by hand.

Table 5-10: IT-1: Row-Level Security blocks out-of-role access

Field	Detail
ID	IT-1
Integration point	App to PostgREST/GoTrue, with RLS policies in Postgres
Scenario	A signed-in client attempts a read or write outside its role, an admin tries to write attendance; a student queries another student's marks
Expected	The database returns only rows the role owns and rejects the out-of-role write; the refusal comes from RLS, not from a client-side check that could be bypassed

Field	Detail
Result	Pass

Table 5-11: IT-2: A write in one app reaches another through sync

Field	Detail
ID	IT-2
Integration point	Two apps and shared Supabase tables and local Room cache (delta sync)
Scenario	A teacher submits marks in one app; a student opens their result in another after the next sync
Expected	The change is pulled on the delta from the stored checkpoint and merged into the second app's cache; the student sees the new result without a manual reload, and a failed sync leaves the checkpoint unchanged so nothing is half-applied
Result	Pass

Table 5-12: IT-3: Privileged operation runs only through an edge function

Field	Detail
ID	IT-3
Integration point	Admin app to Supabase edge function (service-role)
Scenario	An action needing more authority than a signed-in request holds, create an account, change teacher status, promote or archive a session, revoke a student link
Expected	The operation succeeds only through the protected server-side function; the same action attempted as a direct client write is refused, keeping the elevated authority off the client
Result	Pass

5.4 Performance Testing (PT)

Performance here is read against what the design actually tries to make fast, not against a load target. The apps are offline-first: a screen paints from the local Room cache first and refreshes in the background, and a refresh pulls only the rows that changed since the last checkpoint rather than the whole table (the delta sync described in §2.4.3 and covered by the unit tests in §5.1). So the questions worth measuring are whether a cached screen opens without waiting on the network, and whether a refresh moves a small delta rather than everything.

These are indicative local baselines, measured on a single development machine and an Android emulator against representative sample data, a department, an academic session, a full roster, and a set of marks. They are not a benchmarked SLA. Real field figures, physical-device launch time, live college-network latency, and many simultaneous users, need production-sized data, connected devices, and the deployed service, and belong to pilot acceptance rather than being invented here.

Table 5-13: Performance Test Configuration

Item	Setting
Hardware	One development workstation; Android app on an emulator
Sample data	A department, one session, a full student roster, and recorded marks/attendance
What is measured	Cached-screen open, delta-sync payload size, and list responsiveness, as relative observations
Status of figures	Indicative local baseline, not a benchmarked SLA
Explicitly excluded	Physical-device startup, live-network latency, concurrent-user load

Table 5-14: PT-1: Offline-first screen open and delta refresh

Field	Detail
ID	PT-1
Aspect	How quickly a screen becomes usable, and how much a refresh transfers
Setup	Open a data-heavy screen (roster, records) with the cache already populated, then trigger a refresh after changing one record on the server
Observation	The screen renders from the Room cache without waiting on the network; the refresh pulls only the changed row from the stored

Field	Detail
	checkpoint, not the full table, and a failed refresh leaves the last good view in place
Result	Meets the offline-first goal (indicative)

Table 5-15: PT-2: List rendering and repeated navigation

Field	Detail
ID	PT-2
Aspect	UI responsiveness on long lists and back-and-forth navigation
Setup	Scroll a full roster/records list and move repeatedly between screens in a role workspace
Observation	Lists render lazily and stay smooth because the data is already local; moving between screens re-reads from the cache rather than re-fetching, so navigation stays responsive
Result	Meets the responsiveness goal (indicative)

Figure fix-notes (images cannot be edited in this pass):

- **Figure 5-1**, replace with a diagram of the actual measurement setup: dev workstation + Android emulator, the sample dataset, and the app to Room cache to Supabase delta-sync path being observed.
- **Figure 5-2**, replace with a plain summary of the indicative observations (cached-open vs. network-wait, delta payload vs. full-table), labelled as a local baseline, not a benchmarked target chart.

5.5 Summary

Testing was aimed where a fault would hurt most: the shared core logic that every app runs. Automated unit testing is narrow by design, it covers the incremental-sync engine, where a silent paging or checkpoint error would quietly lose or duplicate records (§5.1). The role workflows, sign-in and routing, attendance, marks with the two-person correction, and student linking, were checked as functional scenarios against controlled sample data (§5.2), and the seams where the apps meet Supabase were exercised at the integration level: Row-Level Security refusing out-of-role access, a write in one app reaching another through sync, and privileged actions running only through edge functions (§5.3). Parity between the mobile and desktop builds is held together by the per-screen scenario registry rather than a second test suite. Performance (§5.4) is reported honestly as an indicative local baseline on development hardware, cached screens open without waiting on the network and a refresh moves only changed rows, not a scale or field

measurement. The result is a system whose critical rules are verified, with the limits of that verification stated plainly rather than dressed up.

6. System Conversion

6. Introduction

There is no old system to migrate from, the college runs on paper registers, office files, and word of mouth today, so putting the CMS in place is a fresh rollout rather than a data cutover. That shifts what conversion actually means here: instead of moving records out of a legacy database, the work is standing up a clean backend, seeding it with trustworthy starting data, installing the six apps, and getting people used to a digital routine without disrupting a live term.

The system being rolled out is six apps over one shared backend. Three run on Android (admin, teacher, student, built with Jetpack Compose); three run on Windows (the same three roles, built with Compose Multiplatform on the JVM); and all six talk to a single Supabase project, Postgres through PostgREST, GoTrue for sign-in, Storage for files, edge functions for privileged actions, and Row-Level Security deciding who sees what. A role behaves the same whether someone is on their phone or at a desk, so a user can pick whichever device suits the moment.

This chapter walks through that rollout: the changeover method and why it was chosen (§6.1), deploying the apps and the Supabase project (§6.2), setting up the database schema when there is no legacy data to import (§6.2.1), training users on the real day-to-day flows (§6.2.2), the acceptance checks run after go-live (§6.3), the challenges that came with a multi-platform rollout (§6.4), and a short recap (§6.5).

6.1 Conversion Method

The plan is to start with a pilot in one department and expand from there. That department runs the system on a small, checked set of real records, and its admins, teachers, and students walk the whole journey, sign-in, attendance, marks, fees, notices, before any other department is switched on. Keeping the first group small is the whole point: fewer people to support while everyone is still learning, and any correction lands before it has to be made in a dozen places at once.

The two all-at-once options were weighed and set aside. A direct switch, every department live on day one, would pile record fixes, training, and support onto the same short window with no room to recover from a mistake. Running the paper process fully in parallel with the system would mean staff doing everything twice; the paper registers stay available as a fallback to check against during the pilot, but not as a second live system kept in lockstep. A pilot avoids both traps, and phased expansion then adds departments one at a time, only after the previous stage's issues are resolved and an acceptance check passes.

Table 6-1: Selection of the conversion method

Method	How it works	Fit for CMS
Direct	Everyone moves to the new system at once, paper stops	Rejected, too much risk and support load in one window, no

Method	How it works	Fit for CMS
	immediately	room to recover
Parallel	Paper and the system run fully side by side for a period	Rejected as primary, duplicates all staff effort; paper is kept only as a fallback to check against
Pilot	One department goes live on checked real data first	Chosen for stage one , a small, controlled setting to verify records, access, workflows, and training
Phased	Departments are added one at a time after the pilot	Chosen for expansion , each new department joins only after the prior stage passes its acceptance check

Figure fix-note (image cannot be edited in this pass): **Figure 6-1**, replace with a simple roadmap showing the pilot department going live first, an acceptance-check gate, then departments added one by one in phases.

6.2 Deployment

Deployment has two halves that have to land in the right order: the shared Supabase backend comes up first, because every app is useless without it, and only then does each app get installed on the devices for its role. All six apps read and write the same college records through that one backend, so there is a single source of truth rather than six copies to keep in step.

The apps build from the nine-module Gradle project (settings.gradle.kts): three Android apps (mobile-admin, mobile-teacher, mobile-student) and three Compose-Multiplatform/JVM desktop apps (desktop-admin, desktop-teacher, desktop-student), all sitting on the shared core, mobile-shared, and desktop-shared modules. The Android apps package as APKs through the Android Gradle Plugin; the desktop apps package as Windows MSI installers through Compose Desktop's nativeDistributions. The toolchain is fixed, JDK 17, Kotlin 2.1.0, AGP 8.13.2, Gradle 8.13, so a build is reproducible rather than machine-dependent. One honest caveat: the current mobile artifacts are controlled test builds; signed release packaging and on-site install are part of the pilot, not something the build alone completes.

The backend is a single Supabase project. It carries the Postgres database reached through PostgREST, GoTrue for sign-in and session handling, two Storage buckets (exam-papers and photos), and the edge functions that run the privileged operations. Row-Level Security is what enforces access on every table, so a client can only ever read or write the rows its role owns. The project URL and keys live in protected build configuration, never in the repository.

Table 6-2: Client applications and artifacts

Component	What is deployed	Where / How
Admin / Teacher / Student (mobile)	mobile-admin, mobile-teacher, mobile-student as Android APKs	Android devices; built via AGP from the shared core + mobile-shared modules
Admin / Teacher / Student (desktop)	desktop-admin, desktop-teacher, desktop-student as Windows MSI installers	Windows machines; packaged via Compose Desktop nativeDistributions from core + desktop-shared
Shared modules	core, mobile-shared, desktop-shared	Not deployed on their own, compiled into each app above
Build toolchain	JDK 17, Kotlin 2.1.0, AGP 8.13.2, Gradle 8.13	Fixed in the Gradle version catalog for reproducible builds

Table 6-3: Backend / server components (one Supabase project)

Component	What is deployed	Where / How
Database	Postgres schema, reached over PostgREST	Supabase project; schema applied by the ordered SQL migrations (§6.2.1)
Access control	Row-Level Security policies per table	In Postgres, every read/write is scoped to the caller's role, not the client
Authentication	GoTrue sign-in, sessions, account recovery	Supabase Auth
File storage	exam-papers and photos per-buckets	Supabase Storage, with per-bucket size/type limits
Privileged operations	Edge functions (service-role)	Supabase Functions, create account, change teacher status, promote/archive session, revoke link
Configuration	Project URL and API keys	Protected build config, not committed to the repository

6.2.1 Data Conversion

There is no old database to convert from, the starting records come off paper registers and office files. So "data conversion" here is really two jobs: build an empty, correctly-shaped database, then fill it with checked starting data. The shape comes first and comes from the migrations. The database schema is defined by the ordered SQL migrations under supabase/migrations/ (25 of

them), applied in sequence from the base schema.sql through the Row-Level Security policies and later changes. Because the migrations are ordered and repeatable, the same run rebuilds the same schema every time, which is what makes a fresh project or a restored backup trustworthy.

With the schema in place, staff enter the actual college structure through the Admin apps, there is no legacy import to run, and no documents data to carry over (that feature was removed). Reference data (departments, the current academic session, fee heads) is entered first because everything else references it. Student rosters are the one bulk case: a roll-number-and-name list can be brought in from a CSV or XLSX file, and the parser skips any row missing a roll or name and flags duplicate rolls before the import is accepted. Everything is loaded under the same Row-Level Security that governs normal use, so nothing is written outside the loading admin's authority.

Table 6-4: Data conversion steps and controls

Step	Action	Control / safeguard
1. Build the schema	Apply the 25 ordered migrations to the Supabase project	Migrations are ordered and repeatable, the same run yields the same schema; a failed run stops rather than half-applying
2. Seed reference data	Enter departments, the academic session, and fee heads in the Admin apps	Compare totals against the approved college list before dependent data is added
3. Create staff accounts	Create authorised admin and teacher accounts with role, department, and status	The office confirms identity, role, and active status; privileged creation runs through the edge function, not a direct write
4. Load student rosters	Import a roll+name list from CSV/XLSX, or enter students individually	Rows missing a roll or name are skipped and duplicate rolls flagged before acceptance; the final roster count is reconciled
5. Enter curriculum & timetable	Add courses, teaching assignments, rooms, periods, and weekly schedules	A clash is refused at write time by the database trigger; staff compare a sample and the full timetable against the signed plan
6. Enter opening records	Where needed, enter verified starting attendance and marks before the pilot	Teachers and the department confirm totals and sampled student records; marks stay within their maxima (25/15)

Data protection during load: a managed Postgres backup is taken before loading; afterwards, totals and sampled records are checked against the signed sources, accepted data is backed up, and a serious failure restores the last good backup.

Figure fix-note (image cannot be edited in this pass): **Figure 6-2**, replace with a flow showing: apply migrations to seed reference data to create accounts to import/enter rosters to enter curriculum/opening records, with the backup-before-load and check-against-source control points marked.

6.2.2 Training

Training is organised by role, and by the actual task each person does rather than a tour of every screen. Each session is a short demonstration on pilot records, then supervised practice, then a quick check that the learner can do the task unaided. Mobile and desktop users get the same instructions for the work itself, the rules and decisions are identical, and only the Windows-specific steps differ, mainly how a file is saved or printed. Someone is trained as ready when they can repeat their task without help, recognise the warning and empty-state messages, and know who to contact when a record looks wrong.

Table 6-5: Training coverage

Audience / role	What they are trained on	Walkthrough
Admin (mobile + desktop)	Create people and academic records, set up rosters, manage schedules and fees, review link and mark-correction requests, read dashboard figures	Includes generating the fee challan on desktop (Use Case 2)
Teacher (mobile + desktop)	Open assigned classes, record attendance and marks (midterm /25, sessional /15), raise a mark correction, read schedules and notices	Recording attendance (Use Case 1)
Student (mobile + desktop)	Link the account to a college record, then read personal attendance, marks, timetable, datesheet, fees, and notices	Account linking and reading the fee challan
Desktop-specific steps	Choosing a save location, saving/exporting a record, and printing to a selected printer	The native save/print dialog in Use Case 2

Use Case 1, Record class attendance (*Teacher Mobile and Desktop*)

The same labels and decisions appear in both versions; the wider desktop window only gives more viewing space.

1. Sign in to the Teacher app and open Attendance.
2. Select the assigned class for the date; if none is selected, the app asks the teacher to choose one.
3. Review the class name, date, roster total, and any attendance already recorded.
4. Mark every student Present, Absent, or Leave, setting the late flag or a note where needed, a late student stays Present with the late flag, not a separate status.
5. Confirm every student is marked, submit the register, and open Attendance History to review or check an earlier record.

Use Case 2, Generate and print the fee challan (*Admin Desktop*)

1. Sign in to the Admin Desktop app and open the session's Fees screen.
2. Select the student (or the sample) and generate the challan.
3. The app builds a three-copy challan PDF, student, college, and clerk copies.
4. In the native save dialog, choose where to save it, or send it to a printer; cancelling writes nothing and changes no college data.
5. The saved file opens in the default viewer for a final check before it is printed and issued.

Figure fix-notes (screenshots cannot be edited in this pass, capture from the current Modernist-themed UI):

- **Figure 6-3**, Teacher app: selecting an assigned class for attendance.
- **Figure 6-4**, Teacher app: a completed class register (Present/Absent/Leave with the late flag) ready to submit.
- **Figure 6-5**, Admin Desktop: the session Fees screen with the generate-challan action.
- **Figure 6-6**, the generated three-copy challan PDF (student/college/clerk) in the save/print dialog or default viewer.

6.3 Post Deployment Testing

Once the pilot department is running on real accounts, devices, and the deployed Supabase project, the system is checked again, this time end to end, against things development testing cannot stand in for: real sign-ins, a live network, and actual college records. These are manual acceptance checks, each a plain pass/fail criterion tied to a requirement, and each result is signed off at the college rather than assumed here in advance.

Table 6-6: Pilot post-deployment acceptance checks

Check	Acceptance criterion	Result
Sign-in and role routing (<i>FR-1</i>)	A valid sign-in lands each user in their own role's workspace; a wrong credential is refused	To sign off at pilot
Access scoping (<i>RLS</i>)	Each role sees only the rows it	To sign off at pilot

Check	Acceptance criterion	Result
	owns, a student cannot read another student's records, an admin cannot write attendance, enforced by the database, not the client	
Record attendance (<i>FR-10</i>)	A teacher marks a full register (Present/Absent/Leave, late flag), submits, and the saved register reappears in history	To sign off at pilot
Enter marks and correct one (<i>FR-12/13/14</i>)	Scores save within their maxima (midterm /25, sessional /15) and lock; a correction is applied only after a different administrator approves it	To sign off at pilot
Generate the fee challan (<i>FR-21</i>)	The desktop app produces a correct three-copy challan PDF (student/college/clerk) and saves or prints it through the native dialog	To sign off at pilot
Cross-app sync (<i>FR-24</i>)	A record written in one app appears in another after the next sync, without a manual reload	To sign off at pilot

The checks are deliberately the workflows that matter most day to day, not an exhaustive pass over every screen. Customer-site execution and formal sign-off are pilot activities; the table above is the acceptance procedure to run there, with results recorded against each criterion.

6.4 Challenges

The rollout's hard parts were mostly consequences of decisions made earlier in the project, and they are worth stating plainly rather than smoothing over.

The biggest was keeping six apps consistent. Three mobile and three desktop apps, each for a different role, could easily have drifted into six slightly different interpretations of the same rule. The response was to push the actual logic, validation, sync, the domain model, down into the shared core module so every app runs the same code, and to keep a per-screen parity checklist (the `ParityScenarioRegistry`) that lists the states each screen must handle so mobile and desktop can be checked against the same expectations. The tradeoff is that a change to shared code touches all six apps at once, which is exactly why it needed a checklist rather than trust.

Getting Row-Level Security right was the second. Enforcing access in the database instead of the client is the safer design, but it means every table needs a policy that is neither too open nor too

tight, and a policy mistake fails quietly, you get back fewer rows, not an error. The policies were iterated per table and checked by trying out-of-role reads and writes and confirming the database refused them.

Offline-first sync was the third real tradeoff. Letting apps work from a local cache and reconcile later invites two classic bugs: losing a change, or applying it twice. The checkpoint-and-delta design pulls only what changed since the last watermark and advances that watermark only when the whole sync succeeds, so a failed sync re-pulls cleanly rather than half-applying, the behaviour the unit tests in §5.1 pin down.

The last was human, not technical. There is no legacy data to import, but there are also users who have never used a system like this. A pilot in one department, with training on the real attendance and challan flows, keeps that learning curve small and contained instead of hitting the whole college at once.

Table 6-7: Conversion and deployment challenges

Challenge	Response
Six apps (3 mobile + 3 desktop) drifting into inconsistent behaviour	Shared core logic so all six run the same rules; a per-screen parity checklist (ParityScenarioRegistry) to verify mobile and desktop against the same states
Row-Level Security that is neither too open nor too tight, and fails silently when wrong	Policies iterated per table and checked by attempting out-of-role reads/writes and confirming the database refuses them
Offline-first sync losing or duplicating rows	Checkpoint/delta design, pull only changes since the last watermark, advance it only on full success so a failed sync re-pulls cleanly (verified by the §5.1 unit tests)
No legacy data, and users new to a digital process	Pilot rollout in one department with training on the real attendance and fee-challan flows, keeping the learning curve small

6.5 Summary

Because there is no old database to move from, this chapter treated conversion as a fresh rollout rather than a cutover. It starts with a pilot in one department and expands in phases (§6.1), keeping the first group small enough to support and correct while everyone learns. Deployment brings up the single Supabase backend first, then installs the six Kotlin apps, Android APKs and Windows MSIs built from the shared modules (§6.2). The database schema is laid down by the ordered migrations with checked reference data entered through the apps and rosters imported from CSV/XLSX, all under Row-Level Security (§6.2.1). Training is task-first, built around the flows people actually use, recording attendance and generating the three-copy fee challan (§6.2.2), and the deployed system is then accepted through manual pass/fail checks on the

workflows that matter, signed off at the pilot (§6.3). The real difficulties were named rather than glossed: keeping six apps consistent, getting Row-Level Security right, reconciling offline sync without losing or duplicating data, and moving users onto a digital process for the first time (§6.4).

7. Conclusion

7. Introduction

The point of this chapter is to check the finished system against what it set out to do. The College Management System was built to meet the objectives set in Chapter 1, delivered as six Kotlin apps, three on Android, three on Windows, over one Supabase backend. What follows measures that: the delivered features against the objectives (§7.1), a traceability matrix connecting every requirement to its design, code, and test evidence (§7.2), how each objective maps to the functionality that realises it (§7.3), and the work worth doing next (§7.4).

One distinction runs through the chapter and is worth stating up front: a feature that is built and verified in the code is treated separately from a step that still needs college-site acceptance. The apps and backend exist and work; the pilot deployment, real-user sign-off, and production-scale measurement are the next steps, not claims already banked.

7.1 Evaluation

The objectives set in Section 1.3 were written to be checkable, so this is a direct pass over them against the finished system. Most are met in the sense that matters here, the capability is present and working in the code. Two are honestly "largely met": platform parity, where the shared logic and the parity checklist keep the six apps aligned but a full visual sign-off on field devices is still pilot work; and the testing objective, where the workflows are covered by functional and integration checks but automated coverage is narrow and full device/user acceptance is still to come.

One item that appeared in early drafts is not claimed as delivered: a general document-sharing/upload feature (the withdrawn FR-25/FR-26) was descoped and removed, so it is absent from the objectives below rather than reported as met.

Table 7-1: Evaluation of project objectives

Objective (Section 1.3)	Delivered feature	Met?
Six apps, mobile + desktop for admin, teacher, student, on one shared database	All six build from the nine-module Gradle project as Android APKs and Windows MSIs against one Supabase backend	Met
One authoritative record for every department, session, subject, teacher, student,	Single shared Supabase schema; each value lives in one table, read by all apps	Met

Objective (Section 1.3)	Delivered feature	Met?
timetable, mark, result, and fee		
Teacher records a class attendance register on screen, visible to admin and the student after sync	Attendance capture (Present/Absent/Leave, late as a flag) syncing to the other apps	Met
A recorded mark can only change through an administrator's approval	Lock-then-request correction via mark_edit_requests, approved by a different administrator	Met
Semester results, GPA, CGPA, supply subjects, held against the enrolled student	student_semester_gpa and result records per student	Met
Each student sees their own attendance, timetable, marks, results, challan, fines, datesheets, and notices on their device	Student mobile + desktop academic overview	Met
Publish notices, calendar events, and examination datesheets to the intended roles on both platforms	Audience-scoped publication enforced by Row-Level Security; datesheets have an explicit published state	Met
Live admin summaries drawn from the records, with a teacher's view scoped to their own classes	Admin dashboard/insights; teacher views scoped by the same rules	Met
Role-based access throughout	Row-Level Security in Postgres, the database, not the client, decides what each role sees	Met
Keep each role's mobile and desktop versions consistent	Shared core logic plus the ParityScenarioRegistry checklist	Largely met, full visual sign-off on field devices is pilot work
Verify all six apps through functional and UI testing before submission	Functional + integration scenario checks on the role workflows and the app and Supabase/RLS seams; automated unit coverage on the sync logic	Partially met, coverage is narrow; full device/user acceptance is still to come

7.2 Traceability Matrix

Every functional requirement is traced here to where it is designed, where it lives in the code, and how it was checked. The code column names a representative artifact, a table, a controller, an edge function, or a shared UI component, rather than every file that touches the feature; it is there to show the requirement is real in the codebase, not to walk through the implementation. The test column is honest about method: some requirements have an automated test ID (the sync engine, UT-1..3; the role workflows, FT-1..4; the app and Supabase seams, IT-1..3), and the rest were verified as manual scenario checks against controlled data. Two requirements, FR-25 and FR-26, are kept in the matrix for continuity but marked withdrawn, the document-publishing/download feature was removed, so there is nothing to deliver or test.

Table 7-2: Complete requirements traceability matrix

FR	Design area	Code artifact	Test
FR-1 Sign in & role routing	Auth & routing	core auth + Supabase GoTrue; per-role entry	FT-1, IT-1
FR-2 Manage admin accounts	Account management	create-account edge function; admin controllers	IT-3, scenario
FR-3 Manage departments	Data model	departments + dept controllers	IT-1, scenario
FR-4 Manage academic sessions	Data model lifecycle	/academic_sessions, session_*; promote/archive edge functions	IT-3, scenario
FR-5 Manage curriculum	Data model	session_subjects + curriculum controllers	Scenario check
FR-6 Manage students & rosters	Data model / import	session_students, StudentImportParser, SessionStudentsController	IT-1, scenario (import)
FR-7 Manage teachers & permissions	Account & management	teachers/profiles; change-teacher-status edge function	IT-3, scenario
FR-8 Manage timetables	Scheduling + clash rule	timetable_periods/period_sessions; fn_check_timetable_conflict trigger	Scenario (clash refused)
FR-9 View personal schedule	Read / RLS scope	Timetable read in teacher/student controllers	Scenario; UT-1..3 (sync)
FR-10 Mark attendance	Attendance process	session_attendance MarkAttendanceController (Present/Absent/Leave, late flag)	+FT-2, IT-2

FR	Design area	Code artifact	Test
FR-11 View attendance history	Read / offline	session_attendance read Attendance History	+IT-2; UT-1..3 (sync)
FR-12 Enter assessment marks	Marks process	session_marks MarksEntryController; AcademicEnums (midterm 25 / sessional 15)	+FT-3
FR-13 Request mark correction	Correction workflow	mark_edit_requests MarkEditRequestsController	+FT-3
FR-14 Review mark correction	Correction workflow	mark_edit_requests approval (admin is not author)	FT-3
FR-15 Record semester results	Data model	student_semester_gpa + result records	Scenario check
FR-16 View marks & semester results	Read / offline	Result reads in student controllers	Scenario; UT-1..3 (sync)
FR-17 Submit examination paper	Storage workflow	exam_paper_submissions + exam- papers bucket; ExamPaperSubmissionRepository	Scenario check
FR-18 Build & publish datesheet	Scheduling + clash rule	datesheets/datesheet_slots; fn_check_datesheet_conflict trigger; published state	Scenario (clash + publish)
FR-19 View datesheet	Read / RLS scope	Datesheet read, audience-scoped	Scenario check
FR-20 Manage session fee structure	Fees data model	session_fees, session_fee_heads, fee_overrides	Scenario check
FR-21 View fee challan	Fees / PDF	FeeChallanPdfGenerator (3-copy) + fees screens	Scenario (see §4.1.10)
FR-22 Issue student fine	Fees data model	finer + fine controllers	Scenario check
FR-23 View student fines	Read / RLS scope	finer read, student-scoped	Scenario check
FR-24 Manage calendar events	Publication	calendar_events (RLS audience)	IT-2; UT-1..3 (sync)
FR-25 Publish college documents	N/A	N/A	Withdrawn (feature removed)
FR-26 View & download	N/A	N/A	Withdrawn (feature removed)

FR	Design area	Code artifact	Test
documents			
FR-27 Submit student link request	Linking workflow	student_link_requests LinkRequestsController	+FT-4
FR-28 Review student link request	Linking workflow	student_link_requests approval; revoke-link edge function	FT-4, IT-3
FR-29 Publish notifications	Publication	notifications (RLS audience)	IT-1, scenario
FR-30 View notifications	Read / offline	notifications read + DataStore last seen	Scenario check
FR-31 View insights & reports	Insights	Client-computed insights controllers	Scenario check
FR-32 View & maintain personal profile	Profile	profiles/teachers + photos bucket	Scenario check
FR-33 Desktop file & print services	Desktop platform	RecordsExporter, FeeChallanPdfGenerator, AwtDesktopPlatformServices	PT-2, scenario

7.3 Conclusion

The CMS takes a college running on paper registers and office files and replaces it with six role-based apps, admin, teacher, and student, each on Android and Windows, over one shared Supabase backend. The correlation below lines each objective up against the functionality that actually delivers it.

Table 7-3: Objective and functionality correlation

Objective	Realising functionality	Outcome
One system across roles and devices	Six apps built from shared modules on one backend	Delivered
One authoritative record set	Single Supabase schema read by every app	Delivered
Digital attendance	On-screen register (Present/Absent/Leave, late flag) syncing to the other roles	Delivered
Protected mark changes	Lock-then-approve corrections via mark_edit_requests	Delivered

Objective	Realising functionality	Outcome
Personal academic access	Student overview: attendance, marks, results, timetable, fees, notices	Delivered
Role-scoped publication	Notices, calendar events, datesheets scoped by Row-Level Security	Delivered
Live admin summaries	Dashboard/insights drawn from the records, teacher views scoped	Delivered
Role-based security	Row-Level Security enforced in Postgres, not the client	Delivered
Mobile/desktop consistency	Shared core logic + the parity checklist	Delivered; field visual sign-off is pilot work
Verified before submission	Functional/integration scenario checks + unit coverage on sync	Partial, narrow automated coverage

Two design choices carried most of the weight and are worth naming. Putting the domain logic in a shared core means all six apps compute the same rules from the same code, attendance percentages, marks limits, sync deltas, instead of six subtly different versions; keeping access control in Row-Level Security means a permission is enforced by the database itself, so a client bug cannot hand a user data they should not see. Those two decisions are what let a small team ship six coherent apps rather than six drifting ones.

It is also worth being straight about the limits. Automated testing is narrow, it covers the sync engine well, and the rest was checked as manual scenarios rather than a broad automated suite. Some scope was cut along the way: the document-publishing feature (FR-25/FR-26) was withdrawn rather than half-built. And the pilot deployment, real-user acceptance, and production-scale measurement are still ahead, not banked. What exists is a working, internally consistent system that meets its core objectives; what remains is the field validation that turns "works in the code" into "accepted at the college."

7.4 Future Work

The most useful next steps come from two places: gaps that are genuinely thin today, and scope that was deliberately cut. None of the following is built, this is a prioritised list of what would add the most, not a preview of hidden features.

The clearest gap is testing. Automated coverage today is narrow, it covers the sync engine well and leaves the rest to manual scenario checks, so widening the automated suite over the domain rules (marks limits, attendance completeness, the mark-correction guard) is the highest-value work, because it protects every one of the six apps at once. Real push notifications are the next

honest gap: notifications are currently rows the apps read under Row-Level Security and pick up on sync, not delivered by a push service, so a device only learns of a notice when it next refreshes. Adding true push would make time-sensitive notices land immediately. Beyond that sit the more optional additions, richer reporting and analytics past the current insights, the document-publishing feature that was withdrawn (FR-25/FR-26) if it is ever wanted back, and possibly extending to iOS or web, which the shared core makes plausible but which is a real amount of work.

Table 7-4: Future work, prioritised

Item	Why it matters	Priority
Broader automated test coverage	Only the sync logic is unit-tested today; automating the domain rules would protect all six apps against regressions	High
Real push notifications	Notifications are RLS-scoped rows read on sync, not pushed, a device sees a notice only on its next refresh; push would make time-sensitive notices immediate	High
Richer reporting / analytics	The current insights are useful but basic; deeper trend and cohort reporting would help administrators	Medium
Document publishing (withdrawn FR-25/FR-26)	Cut from this version; worth reconsidering only if a concrete need for shared college documents appears	Low
iOS / web clients via shared core	The shared logic makes another platform feasible, but it is a substantial build for an audience not yet asked for	Low

Appendix A: Fully Dressed Use Case Descriptions

These are the twenty principal use cases in fully dressed form. They are the same set summarised in Section 2.2.3 and are kept identical to it, Appendix A is the detailed reference, not a second, diverging version. Each names the responsible actor, the applications that support it, the requirements it traces to, the normal sequence, the exceptions worth planning for, and the rules that keep the records trustworthy. There is no document-publishing or document-download use case: that feature (FR-25/FR-26) was withdrawn, so publishing (UC-07) and reading announcements (UC-18) cover notices, calendar events, and datesheets only.

UC-01: Sign In and Open the Correct Application Area (*Table A-1*)

Field	Detail
Requirement	FR-1
Purpose	Let a registered user sign in and land in the area that matches their role.
Priority	High
Applications	All six
Actors	Administrator, Teacher, Student
Description	A registered user signs in and is taken to the correct role-based home. A still-valid session is reused when the app reopens.
Trigger	The user opens an app or submits the sign-in form.
Preconditions	The user has a registered account; admin and teacher accounts are active.
Postconditions	The right home area is shown and only that role's actions are available.
Normal Flow	1. The user opens the app. 2. It checks for a valid session. 3. If needed, the user enters email and password. 4. The account and role are verified. 5. The role's home opens.
Exceptions	Wrong details give a clear message; a disabled account is refused; a role mismatch routes the user to the right app; a connection failure shows a short retry note without exposing service addresses.
Business Rules	A user reaches only the role and records their account is assigned.

Field	Detail
Includes / Extends	Included by every protected use case.

UC-02: Manage Administrator Accounts *(Table A-2)*

Field	Detail
Requirement	FR-2
Purpose	Let an existing administrator add and maintain other authorised administrators.
Priority	High
Applications	Admin Mobile, Admin Desktop
Actors	Administrator
Description	An administrator views the administrator directory, creates an account, and maintains its college identity and status.
Trigger	The administrator opens People to Administrators.
Preconditions	Signed in as an active administrator.
Postconditions	The directory reflects the addition or update.
Normal Flow	1. Open the directory. 2. Existing administrators are listed. 3. Select Add Administrator. 4. Enter identity and contact details. 5. The system validates and creates the account. 6. The directory refreshes.
Exceptions	Missing, invalid, or duplicate details are flagged by the field; the last usable administrative access cannot be removed by accident; a service failure leaves the directory unchanged.
Business Rules	Only an administrator may create or maintain administrator accounts; email and identity are unique.
Includes / Extends	Includes UC-01.

UC-03: Manage Academic Structure *(Table A-3)*

Field	Detail
Requirement	FR-3, FR-4, FR-5

Field	Detail
Purpose	Maintain departments, academic sessions, semesters, and subjects, the foundation every academic record hangs off.
Priority	High
Applications	Admin Mobile, Admin Desktop
Actors	Administrator
Description	The administrator creates and reviews departments, sessions, semester progress, and the subjects taught in each semester.
Trigger	The administrator opens Academics.
Preconditions	Signed in. A department must exist before a session, and a session before its subjects.
Postconditions	The structure is available to timetable, attendance, marks, fee, and reporting work.
Normal Flow	1. Select or create a department. 2. Create or open a session. 3. Review semester and session detail. 4. Add subjects to the right semester. 5. The saved structure is confirmed.
Exceptions	Duplicate identifiers, invalid years, missing names, or incomplete semester detail are rejected with field-level guidance; deleting something other records depend on is blocked or confirmed.
Business Rules	Every session belongs to one department; every subject belongs to a defined session and semester.
Includes / Extends	Includes UC-01.

UC-04: Manage Teachers and Students (Table A-4)

Field	Detail
Requirement	FR-6, FR-7
Purpose	Maintain the people tied to each academic session and the responsibilities granted to teachers.
Priority	High

Field	Detail
Applications	Admin Mobile, Admin Desktop
Actors	Administrator
Description	The administrator adds or imports students, reviews student profiles, creates teacher accounts, and manages teacher status and permissions.
Trigger	The administrator opens People, a session roster, or a student profile.
Preconditions	Signed in; the relevant department and session exist.
Postconditions	Valid teacher and student records are available to authorised workflows.
Normal Flow	1. Open Teachers or a session's Students. 2. Existing records show. 3. Add, import, or update the person. 4. Identity and session information are validated. 5. The list refreshes with the saved record.
Exceptions	Duplicate roll numbers or emails are rejected; invalid import rows are reported without hiding the good ones; disabling a teacher is confirmed and keeps their history.
Business Rules	A roll number is unique within its session; teacher permissions are granted only by an administrator.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-05: Prepare and Maintain Timetables *(Table A-5)*

Field	Detail
Requirement	FR-8
Purpose	Build a weekly schedule without booking the same teacher or room into overlapping periods.
Priority	High
Applications	Admin Mobile, Admin Desktop
Actors	Administrator, authorised Teacher

Field	Detail
Description	An authorised user creates or updates class periods for sessions, subjects, teachers, rooms, days, and times.
Trigger	The user opens the Master Timetable or a session timetable.
Preconditions	Relevant sessions, subjects, teachers, and rooms exist; the user has timetable permission.
Postconditions	The timetable becomes available to the affected administrators, teachers, and students.
Normal Flow	1. Open the timetable. 2. Pick a day and period. 3. Enter session, subject, teacher, room, and time. 4. The system checks for clashes. 5. The period is saved and the timetable refreshes.
Exceptions	A teacher, room, or session clash blocks the save and names the conflict; incomplete times or missing assignments are rejected; cancelling changes nothing.
Business Rules	A teacher, room, or session cannot hold two overlapping periods (enforced by the database's own no-double-booking constraint).
Includes / Extends	Includes UC-01.

UC-06: Review Approval Requests (Table A-6)

Field	Detail
Requirement	FR-14, FR-28
Purpose	Give authorised staff a controlled way to approve student account links and corrections to locked marks.
Priority	High
Applications	Admin Mobile, Admin Desktop; Teacher Mobile/Desktop for link requests when permitted
Actors	Administrator, authorised Teacher
Description	The reviewer opens a pending request, compares it with college records, and approves

Field	Detail
	or rejects it with a recorded outcome.
Trigger	The reviewer opens a pending request from People or Requests.
Preconditions	A pending request exists and the reviewer has permission.
Postconditions	The request is approved or rejected; an approval is applied to the student link or mark.
Normal Flow	1. Open the request queue. 2. Select a pending request. 3. Compare submitted and existing information. 4. Choose Approve or Reject. 5. Confirm; the decision is recorded.
Exceptions	The reviewer may leave without deciding; a decided request cannot be decided again; a rejection reason is required where asked; a failure leaves the request pending.
Business Rules	Whoever submitted a mark-correction request cannot approve it; each request gets exactly one final decision.
Includes / Extends	Includes UC-01; may extend UC-11 or UC-14.

UC-07: Publish College Information (Table A-7)

Field	Detail
Requirement	FR-18, FR-20, FR-22, FR-24, FR-29
Purpose	Create and publish datesheets, fees, fines, calendar events, and notifications for the intended users.
Priority	High
Applications	Admin Mobile, Admin Desktop; permitted Teacher apps for selected notices and datesheets
Actors	Administrator, authorised Teacher
Description	An authorised user prepares college information, chooses its audience or academic scope, reviews it, and publishes it to the relevant apps.
Trigger	The user selects an Add, Create, Issue, or

Field	Detail
	Publish action.
Preconditions	Signed in with the required permission; any referenced session or student exists.
Postconditions	The item is stored and becomes visible to its intended users according to its publication status and audience.
Normal Flow	1. Open the relevant records area. 2. Enter the content and dates. 3. Choose audience and academic scope. 4. The system validates. 5. The user confirms publication. 6. The published item appears in the list.
Exceptions	Missing dates, invalid amounts, or incomplete audience information prevent publication; a datesheet clash is refused by the database; the user may save or cancel where allowed.
Business Rules	Only authorised users publish; students see only what is addressed to them or their group; a fine must name a student and amount.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-08: Review Reports and Insights *(Table A-8)*

Field	Detail
Requirement	FR-10, FR-11, FR-12, FR-15, FR-16, FR-31
Purpose	Help authorised staff review attendance, academic progress, and college summaries without stitching registers together by hand.
Priority	Medium
Applications	Admin Mobile, Admin Desktop, Teacher Mobile, Teacher Desktop
Actors	Administrator, Teacher
Description	The user opens a report or insight, applies the available academic filters, and reads totals, trends, or students needing attention, within their own authority.

Field	Detail
Trigger	The user opens Attendance Records, Insights, or an academic summary.
Preconditions	Signed in; relevant academic records exist.
Postconditions	The report is displayed; viewing changes no records.
Normal Flow	1. Open a reporting area. 2. Available scope and filters show. 3. Pick a department, session, semester, subject, or period where relevant. 4. The authorised summary is prepared. 5. The user reviews or exports it.
Exceptions	No matching records gives an informative empty state; invalid filters are ignored or corrected; a loading failure offers retry.
Business Rules	Administrators see college-wide information; teachers see only their assigned records, the same scoping the database enforces.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-09: Mark Class Attendance *(Table A-9)*

Field	Detail
Requirement	FR-10
Purpose	Record one class meeting's attendance accurately for every enrolled student.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher picks an assigned class and date, marks each student, checks the totals, and submits the register.
Trigger	The teacher selects Mark Attendance.
Preconditions	Signed in, assigned to the class, roster available.
Postconditions	One attendance record is stored per student and becomes visible to authorised users.

Field	Detail
Normal Flow	1. Select an assigned class. 2. The date and roster show. 3. Mark each student present, absent, or on leave, adding the late flag where needed. 4. Review the totals. 5. Submit; confirmation shows.
Exceptions	An incomplete register is caught before submission; a duplicate register opens for review rather than silently duplicating; a connection failure keeps the screen and offers retry.
Business Rules	A teacher marks only assigned classes; one final status per student per meeting, with late recorded as a flag on a present student. The admin can read attendance but never writes it.
Includes / Extends	Includes UC-01.

UC-10: Enter Assessment Marks *(Table A-10)*

Field	Detail
Requirement	FR-12
Purpose	Record valid marks for students in an assigned subject and assessment.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher picks a class and assessment, enters each student's mark, clears any validation messages, and submits the list.
Trigger	The teacher selects Marks Entry.
Preconditions	Assigned to the subject; roster and the allowed maximum exist.
Postconditions	Valid marks are stored and become read-only to the teacher unless a correction is approved.
Normal Flow	1. Select session, subject, semester, and assessment type. 2. The roster shows. 3. Enter marks. 4. The system checks values and completeness. 5. Submit. 6. Confirmation and

Field	Detail
	locked status show.
Exceptions	Negative marks, values above the maximum, and missing entries are rejected; an already-submitted list opens locked; a failed submission keeps the entries for retry.
Business Rules	Marks stay within the permitted range (midterm out of 25, sessional out of 15); a submitted mark cannot be changed directly.
Includes / Extends	Includes UC-01; may be extended by UC-11.

UC-11: Request a Mark Correction (Table A-11)

Field	Detail
Requirement	FR-13
Purpose	Let a teacher request a justified correction instead of editing a locked mark directly.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher selects a locked mark, enters the proposed value and a reason, and submits it for administrative review.
Trigger	The teacher selects the correction action beside a locked mark.
Preconditions	The teacher owns or is responsible for the mark, and it is locked.
Postconditions	A pending request reaches an administrator while the original mark stays unchanged.
Normal Flow	1. Open the locked award list. 2. Select a student's mark. 3. Enter the proposed mark and reason. 4. The system validates the range and checks for another pending request. 5. Submit; pending status shows.
Exceptions	An invalid proposed mark or missing reason blocks submission; a second pending request for the same mark is refused; cancelling changes

Field	Detail
	nothing.
Business Rules	Only an authorised reviewer applies the change, and never the teacher who requested it; the original value stays active until approval.
Includes / Extends	Extends UC-10; may lead to UC-06.

UC-12: Submit Examination Paper and Semester Results *(Table A-12)*

Field	Detail
Requirement	FR-15, FR-17
Purpose	Let a teacher submit required examination material and record final semester outcomes for assigned students.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher sets the academic context, uploads an examination paper or records semester results (GPA/CGPA and supply subjects), and submits.
Trigger	The teacher opens Submit Exam Paper or Semester Results.
Preconditions	Assigned to the session or subject; the required students and semester information exist.
Postconditions	The paper or result is stored and available to authorised users according to its status.
Normal Flow	1. Select the session and semester. 2. For a paper, pick the subject and file; for results, enter each student's result. 3. The system validates completeness. 4. Review and submit. 5. Confirmation shows.
Exceptions	Unsupported or missing files, incomplete results, and invalid academic values block submission; cancelling changes nothing; a transfer failure offers retry.
Business Rules	Teachers submit only for assigned work; result

Field	Detail
	values follow the college's grading rules.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-13: View Teaching Schedule and Assigned Students (*Table A-13*)

Field	Detail
Requirement	FR-6, FR-9
Purpose	Give a teacher a reliable view of assigned classes, periods, and students.
Priority	High
Applications	Teacher Mobile, Teacher Desktop
Actors	Teacher
Description	The teacher reviews the weekly schedule, the next class, assigned sessions, and the student rosters the timetable permits.
Trigger	The teacher opens Home, Schedule, or My Students.
Preconditions	Signed in with at least one assignment.
Postconditions	The schedule or roster is shown without change.
Normal Flow	1. Open the area. 2. The teacher's assignments are identified. 3. Classes or students are grouped by schedule and session. 4. Select an item. 5. Details show.
Exceptions	No assignment gives a clear empty state; missing information offers refresh; the teacher cannot open an unrelated roster.
Business Rules	Teachers see only students and schedules tied to their assigned work.
Includes / Extends	Includes UC-01.

UC-14: Register and Link a Student Account (*Table A-14*)

Field	Detail
Requirement	FR-27, FR-28

Field	Detail
Purpose	Connect a student's sign-in account to the correct college roll-number record, after review.
Priority	High
Applications	Student Mobile, Student Desktop; reviewed in Admin apps and permitted Teacher apps
Actors	Student, Administrator, authorised Teacher
Description	A student registers, enters identifying college information, submits a link request, and waits for a decision before their records open.
Trigger	A new or unlinked student selects Register or Link College Record.
Preconditions	A valid email and an existing college enrolment record.
Postconditions	A pending request is created; on approval the account links to one student record, on rejection the reason or status shows.
Normal Flow	1. The student registers or signs in. 2. The account is recognised as unlinked. 3. The student enters roll number and identity details. 4. The form is validated and a request submitted. 5. A reviewer verifies it via UC-06. 6. The student sees the outcome and gains access on approval.
Exceptions	Invalid details, an unknown roll number, or an already-linked record block submission; a duplicate pending request is not created; rejected requests can be corrected and resubmitted where allowed.
Business Rules	One account links to one student record, and one student record has one approved account.
Includes / Extends	Includes UC-01 and UC-06.

UC-15: View Personal Attendance and Timetable (*Table A-15*)

Field	Detail
Requirement	FR-9, FR-11

Field	Detail
Purpose	Give a linked student a current view of their class schedule and attendance history.
Priority	High
Applications	Student Mobile, Student Desktop
Actors	Student
Description	The student reviews the weekly timetable, next class, attendance totals, per-subject percentages, and available history for their linked record.
Trigger	The student opens Attendance or Timetable.
Preconditions	Signed in and linked to a college record.
Postconditions	The personal schedule or attendance is shown without change.
Normal Flow	1. Open the area. 2. The linked session and roll number are identified. 3. Timetable or attendance loads. 4. Select a day, subject, or summary. 5. Details show.
Exceptions	No published timetable or attendance gives an informative empty state; a connection failure offers retry; the student cannot select another student's record.
Business Rules	A student sees only their own attendance and their linked session's timetable.
Includes / Extends	Includes UC-01.

UC-16: View Marks, Results, and Datesheets *(Table A-16)*

Field	Detail
Requirement	FR-16, FR-19
Purpose	Let a linked student review their assessment information and the published examination schedules.
Priority	High
Applications	Student Mobile, Student Desktop
Actors	Student
Description	The student views subject marks, semester

Field	Detail
	results and progression, and datesheets published for their linked session.
Trigger	The student opens Exams, Marks, Results, or Datesheets.
Preconditions	Signed in and linked; the relevant records exist or are published.
Postconditions	The academic information is shown without allowing student changes.
Normal Flow	1. Open Exams. 2. Select Marks, Results, or Datesheets. 3. Information loads for the linked record and session. 4. Available detail shows. 5. The student reviews it.
Exceptions	Unpublished or unavailable information gives a clear message; incomplete history shows only where valid; a loading failure offers retry.
Business Rules	A student sees only personal marks and results, and only datesheets published for the relevant audience.
Includes / Extends	Includes UC-01.

UC-17: View Fee Challan and Fines *(Table A-17)*

Field	Detail
Requirement	FR-21, FR-23
Purpose	Give a student current fee information without a routine trip to the accounts office.
Priority	High
Applications	Student Mobile, Student Desktop
Actors	Student
Description	The student opens the fee area to review the session fee structure, their challan, due dates, the payment note, and any recorded fines.
Trigger	The student selects Fee Challan or Fines.
Preconditions	Signed in and linked; a fee structure or fine record exists where applicable.

Field	Detail
Postconditions	The fee and fine information is shown without recording any payment.
Normal Flow	1. Open the fee area. 2. The linked session and roll number are identified. 3. Fee heads, totals, due information, and fines are prepared. 4. The student reviews them. 5. The challan can be opened, saved, or printed as a PDF.
Exceptions	No fee structure or fines gives an informative empty state; missing information is never replaced with an invented amount; a loading failure offers retry.
Business Rules	A student sees only their own fee and fine records; showing a challan does not mark it paid.
Includes / Extends	Includes UC-01; may include UC-19 on desktop.

UC-18: View Notices, Events, and Datesheets *(Table A-18)*

Field	Detail
Requirement	FR-24, FR-30
Purpose	Give users one place to read college announcements, calendar events, and published datesheets meant for them.
Priority	Medium
Applications	All six
Actors	Administrator, Teacher, Student
Description	The user opens notifications, the calendar, or datesheets and reads the items addressed to their role or academic group.
Trigger	The user selects a notice, event, or datesheet from the relevant area or a notification entry.
Preconditions	Signed in; the item has been published for the user's audience.
Postconditions	The item is shown and may be marked read where supported; the published content is

Field	Detail
	unchanged.
Normal Flow	1. Open the relevant area. 2. Authorised published items are listed. 3. Select an item. 4. Its detail is displayed. 5. The user returns to the list.
Exceptions	An expired, removed, or unavailable item gives a safe message; empty lists explain that nothing has been published.
Business Rules	Only items addressed to the user's role, session, or the general college audience are shown.
Includes / Extends	Includes UC-01.

UC-19: Use Desktop File, Export, and Print Actions *(Table A-19)*

Field	Detail
Requirement	FR-33
Purpose	Let desktop users complete role-appropriate file, export, and print actions through the familiar operating-system services.
Priority	Medium
Applications	Admin Desktop, Teacher Desktop, Student Desktop
Actors	Administrator, Teacher, Student
Description	A desktop user picks a save location, opens a saved file, prints available information, or exports from an existing workflow (for example, exporting a report or printing a challan).
Trigger	The user selects Open, Save, Export, Print, or Share where available.
Preconditions	Signed in, authorised for the underlying record, using a desktop app.
Postconditions	The chosen operating-system action completes, or returns a clear cancellation or failure to the app.
Normal Flow	1. Open an authorised record or form. 2. Select

Field	Detail
	a desktop action. 3. The app opens the matching system dialog (save, open, or print). 4. The user completes the system step. 5. The app confirms or returns to the screen.
Exceptions	The user may cancel without changing data; missing file access, an unsupported file type, or no printer gives a clear message; internal service details are never exposed.
Business Rules	Desktop actions never bypass the user's role or record permissions; native system windows may look different from the app.
Includes / Extends	Extends UC-04, UC-07, UC-08, UC-12, or UC-17.

UC-20: View and Maintain Personal Profile *(Table A-20)*

Field	Detail
Requirement	FR-32
Purpose	Let each signed-in user review their account identity, update the profile fields they are allowed to, and sign out safely.
Priority	Medium
Applications	All six
Actors	Administrator, Teacher, Student
Description	The user opens their profile, reviews account and college identity, edits allowed contact or profile details, and may sign out.
Trigger	The user selects Profile or Sign Out.
Preconditions	Signed in.
Postconditions	Valid permitted changes are stored, or the session ends and protected information is no longer reachable.
Normal Flow	1. Open Profile. 2. Account and college identity show. 3. Edit an allowed field if needed. 4. The system validates and saves. 5. Return or sign out. 6. Sign-out returns to the authentication

Field	Detail
	screen.
Exceptions	Invalid details are flagged without discarding valid ones; protected identity and role fields stay read-only; a failed save keeps the previous profile; cancelling sign-out keeps the session active.
Business Rules	Users change only fields permitted for their role; role, status, and protected college identity need an authorised administrative process.
Includes / Extends	Includes UC-01.

Appendix B: General Coding Standards and Guidelines

The codebase follows a small set of conventions consistently, and they are worth stating as they actually appear in the source rather than as generic best-practice. The whole project is Kotlin under a single `com.mbd.*` package root, `cmscommon` for the shared code, and `cmsadmin/cmsteacher/cmsstudent` (with their desktop counterparts) for the apps. The architecture is layered MVVM with a repository boundary: the domain model, DTOs, and repositories live in the shared modules, screens observe view-models or controllers, and nothing in the UI reaches the database directly. Access control is deliberately not a coding standard the UI enforces, it is enforced by Row-Level Security in the database, so a screen that forgets a check still cannot read rows the role does not own.

Naming is regular enough to navigate by: types are `PascalCase`, members `camelCase`, and files carry their role as a suffix, `*Screen`, `*ViewModel`, `*Controller`, `*Repository`/`*RepositoryImpl`, `*Dao`, `*Dto`. Asynchronous work uses coroutines and Flow rather than callbacks, DTOs are `kotlinx.serialization` classes, local storage is Room entities and DAOs, and the UI is Jetpack Compose sharing one theme (`ModernistColors` in `Theme.kt`) across every app so the six look like one product.

Table B-1: Coding standards in force

Area	Standard	Example from the codebase
Package layout	One <code>com.mbd.*</code> root; shared code in <code>cmscommon</code> , apps in <code>cms{admin,teacher,student}</code> + desktop variants	<code>com.mbd.cmscommon.controller</code> , <code>com.mbd.cmsadmin.feature.*</code>
Architecture	Layered MVVM + repository; per-domain/DTOs/repos in shared modules; UI never hits the DB directly	core domain + repositories, per-feature <code>*ViewModel</code> / <code>*Controller</code>
Naming	<code>PascalCase</code> types, <code>camelCase</code> members, role suffix on files	<code>MarksEntryController</code> , <code>TeacherRepositoryImpl</code> , <code>SessionStudentsScreen</code>
Dependency injection	Hilt on Android, its Dagger core on desktop	<code>@HiltViewModel</code> on Android view-models; <code>@Inject</code> constructors throughout
Asynchronous code	Coroutines + Flow/StateFlow, suspend functions, no callback chains	repositories expose <code>suspend fun</code> ; view-models expose <code>StateFlow</code>
Data transfer	<code>kotlinx.serialization</code> DTOs for the PostgREST JSON	<code>@Serializable</code> DTO classes in core
Local storage	Room entities and DAOs	<code>@Entity</code> / <code>@Dao</code> types feeding the

Area	Standard	Example from the codebase
	behind the repositories	offline cache
UI	Jetpack Compose with one shared theme across all six apps	@Composable screens; ModernistColors in Theme.kt
Security	Enforced by Row-Level Security in the database, not by the client alone	RLS policies per table; the UI relies on, not replaces, them

Appendix C: Application Prototype

The screenshots in this appendix show the working system across its six apps, three on Android (admin, teacher, student, built with Jetpack Compose) and three on Windows (the same roles, built with Compose Multiplatform). All six share one visual language, the Modernist theme defined in Theme.kt, so the same palette, type, and component shapes carry across phone and desktop; what changes between them is layout density, not identity. The figures below are chosen to cover the work each role actually does rather than every screen, one or two representative captures per role, plus the desktop fee-challan output.

The captures should be taken from the current builds. Each figure lists what it should show and a recapture note, because the images themselves cannot be regenerated in this pass.

Figure C-1: Admin dashboard (Admin Mobile). The administrator's home: live summary figures for departments, sessions, teachers, and students, drawn straight from the records. *Fix-note: REPLACE screenshot with a current capture of the dashboard/home screen from Admin Mobile.*

Figure C-2: Session detail and fees (Admin Desktop). A session opened in the wider desktop layout, showing its students and the fee structure area. *Fix-note: REPLACE screenshot with a current capture of a session's detail/fees screen from Admin Desktop.*

Figure C-3: Mark class attendance (Teacher Mobile). A class register with each student marked Present, Absent, or Leave and the late flag where it applies, ready to submit. *Fix-note: REPLACE screenshot with a current capture of the attendance screen from Teacher Mobile.*

Figure C-4: Marks entry (Teacher Desktop). The roster for one subject and assessment, with scores entered within their maxima (midterm out of 25, sessional out of 15) before submission locks them. *Fix-note: REPLACE screenshot with a current capture of the marks-entry screen from Teacher Desktop.*

Figure C-5: Student academic overview (Student Mobile). The student home bringing together attendance, next class, marks, results, and notices for the linked record. *Fix-note: REPLACE screenshot with a current capture of the academic-overview/home screen from Student Mobile.*

Figure C-6: Fee challan (Admin/Student Desktop). The generated three-copy challan PDF (student, college, and clerk copies) opened in the viewer before printing. *Fix-note: REPLACE screenshot with a current capture of the generated fee-challan PDF from the desktop app.*

There is no documents screen in this set, the document-publishing feature was removed, and no screen should be captured for it.